كلية هندسة الذكاء الاصطناعي
FACULTY OF ARTIFICIAL INTELLIGENCE ENGINEERING
الجامعة السورية الخاصة
SYRIAN PRIVATE UNIVERSITY

AI PROJECT MANGER

(Graduation Project2) prepared to obtain the degree of B.Sc. **in the Department of Artificial Intelligence Engineering and Data Science**

Authors:

أحمد محمد باسل الطباع **محمد عدنان لحام**

Supervised by:

د.اصف جعفر

Damascus: **1447–1448هـ / 2025–2026م**

SUPERVISOR CERTIFICATION

I certify that the preparation of this project, entitled
“AI project manager,”
prepared by
“أحمد محمد باسل الطباع و محمد عدنان لحام”
was carried out under my supervision at the Faculty of Artificial Intelligence Engineering in partial fulfillment of the requirements for the degree of **B.Sc.** in the Department of Artificial Intelligence Engineering and Data Science.

Name:

Date:

Signature:

الملخص

تُعدّ عملية تحويل المتطلبات البرمجية المكتوبة باللغة الطبيعية إلى خطط مشاريع عملية ومنظمة من المسائل الصعبة في هندسة البرمجيات، نظرًا لما تتسم به هذه المتطلبات من غموض، وتفاوت في مستوى التفصيل، وتداخل بين الجوانب الوظيفية وغير الوظيفية. وتزداد أهمية هذه المسألة لأن جودة التخطيط المبكر تؤثر مباشرةً في تقدير الجهد، وتنظيم التنفيذ، وتقليل المخاطر، ورفع احتمالية نجاح المشروع. إلا أن الطرق التقليدية مثل نماذج التقدير العددية أو الاعتماد الكامل على الخبرة البشرية تعاني من محدودية في تفسير النصوص الحرة، بينما تعاني المقاربات المعتمدة على النماذج اللغوية الكبيرة من مشكلات مثل الهلوسة، وعدم الاتساق، وضعف القابلية للتحقق البنيوي، مما يبرز الحاجة إلى إطار هجين أكثر موثوقية وقابلية للتفسير.

يقترح هذا البحث إطارًا هجينًا ذكيًا باسم AI PM لتحويل أوصاف المشاريع البرمجية إلى مخرجات تخطيطية قابلة للتنفيذ. يعتمد الحل على Pipeline متعدد المراحل ومتعدد الوكلاء يدمج بين الاستدلال المعتمد على نموذج لغوي محلي لتفسير المتطلبات وتفكيكها، وقاعدة معرفة متجهية لتوفير دعم سياقي واسترجاع أمثلة مشابهة، وآليات تحقق حتمية لضبط البنية والكشف عن الأخطاء، ونمذجة بيانية للاعتماديات لتمثيل تسلسل التنفيذ والمسار الحرج، وطبقة تقييم متعددة المقاييس لقياس جودة المخرجات بشكل شامل. وتكمن القيمة المضافة للنظام في أنه لا يكتفي بإنتاج قائمة مهام، بل يولد خطة مترابطة تتضمن المهام، الاعتماديات، التقديرات، تنظيم السبرنتات، تقارير النقد والمخاطر، مع الحفاظ على التتبع والشفافية وقابلية المراجعة.

أظهرت نتائج التقييم على 20 عينة Benchmark أن النموذج المقترح حقق نسبة نجاح 100% في إنتاج خطط مقبولة نهائيًا، مع Coverage = 1.000، و0.990 = Dependency Coherence، و0.986 = Critic Score، ومتوسط Overall Score = 0.770. وعلى المستوى الكمي، حقق النظام MMRE = 0.301 و0.691 = PRED(25)، كما بلغ F1-FR = 0.573 و0.482 = F1-NFR، مما يدل على أداء قوي في الجوانب البنيوية مع بقاء مجال للتحسين في الاستقرار الدلالي والتقدير الكمي. وتخلص الدراسة إلى أن المعمارية الهجينة المقترحة تمثل مقاربة فعالة وموثوقة للتخطيط الآلي للمشاريع البرمجية، وأن دمج الاستدلال اللغوي مع التحقق الحتمي والتحليل البياني يحقق توازنًا أفضل بين الذكاء والاعتمادية مقارنةً بالمقاربات التقليدية أو التوليدية البحتة.

. **الكلمات المفتاحية:** التخطيط الآلي للمشاريع البرمجية، النماذج اللغوية الكبيرة، الأنظمة متعددة الوكلاء، استرجاع المعرفة، تحليل الاعتماديات، تقدير الجهد البرمجي.

Abstract

Transforming natural-language software requirements into actionable and well-structured project plans remains a difficult problem in software engineering due to ambiguity, uneven specification detail, and the intermixing of functional and non-functional concerns. The importance of this problem stems from the fact that early planning quality directly affects effort estimation, execution organization, risk reduction, and overall project success. Traditional approaches, such as numerical estimation models or purely expert-driven planning, are limited in their ability to interpret free-form requirements, while LLM-based approaches often suffer from hallucination, inconsistency, and weak structural verifiability. This creates a clear need for a more reliable and interpretable hybrid planning framework.

This thesis proposes AI PM, a hybrid intelligent framework for converting software project descriptions into execution-oriented planning artifacts. The proposed solution is built as a multi-stage, multi-agent pipeline that combines a local large language model for requirement interpretation and task decomposition, a vector knowledge base for contextual grounding and similarity-based retrieval, deterministic validation mechanisms for structural control and error detection, graph-based dependency modeling for execution sequencing and critical-path analysis, and a multi-metric evaluation layer for comprehensive quality assessment. The added value of the framework lies in the fact that it does not merely generate a task list; instead, it produces an integrated and reviewable planning structure that includes tasks, dependencies, estimates, sprint organization, critic reports, and risk reports while preserving traceability and transparency.

Experimental evaluation on 20 benchmark samples showed that the proposed model achieved a 100% success rate in producing final acceptable plans, with Coverage = 1.000, Dependency Coherence = 0.990, Critic Score = 0.986, and an average Overall Score = 0.770. Quantitatively, the system obtained MMRE = 0.301 and PRED(25) = 0.691, while reaching F1-FR = 0.573 and F1-NFR = 0.482, indicating strong structural performance with remaining room for improvement in semantic stability and estimation accuracy. The study concludes that the proposed hybrid architecture constitutes an effective and trustworthy approach to automated software project planning, and that combining language-based reasoning with deterministic validation and graph analytics provides a better balance between intelligence and reliability than either traditional or purely generative methods alone.

Keywords: automated software project planning, large language models, multi-agent systems, retrieval-augmented knowledge, dependency analysis, software effort estimation.

Table of contents

Index of tables

Table of figures

Table of Equations

No.	Equation Name	Mathematical Expression	Purpose in the Thesis
1	Requirement Set Representation	$(R = r_1, r_2, r_3, \dots, r_n)$	Represents the normalized set of software requirements extracted from the input brief or template.
2	Task Representation Tuple	$\left(t_i = (id_i, title_i, type_i, c_i, d_i, h_i, dep_i, s_i)\right)$	Defines the structured representation of each generated

			task, including identity, type, complexity, effort, dependencies, and source trace.
3	Architecture Component Set	$(\mathcal{A} = I, P, L, K, C, E, G, S, R, M, O)$	Formally represents the major components of the proposed AI PM architecture.
4	General Pipeline Composition	$(\Pi = \mathcal{H} \circ \mathcal{G} \circ \mathcal{V} \circ \mathcal{E} \circ \mathcal{P} \circ \mathcal{A})$	Describes the ordered composition of the major pipeline stages: analysis, planning, estimation, verification, graph construction, and enrichment.
5	Sequential Data-Flow Transformation	$\left(D_{out} = M_n\left(M_{n-1}(\dots M_2(M_1(D_{in}))\dots)\right)\right)$	Expresses the staged transformation of raw input data into final planning outputs through multiple modules.
6	Hybrid Planning Function	$\left(P = f(R, K, M, V)\right)$	Represents planning as a function of normalized requirements, retrieved knowledge, model

			reasoning, and validation constraints.
7	Initial Planning Output	$(T_0 = \mathcal{P}(R, K, L))$	Defines the first generated task plan as the output of the planning stage.
8	Estimation-Enriched Plan	$(T_1 = \mathcal{E}(T_0, K))$	Represents the task plan after effort and execution metadata have been added.
9	Verification Result	$(V_r = \mathcal{V}(T_1))$	Represents the output of the verification stage after critic-based review.
10	Graph Construction Output	$(G = \mathcal{G}(T_1))$	Represents the dependency graph built from the validated task set.
11	Final Enriched Output	$(Y = \mathcal{H}(T_1, G, V_r))$	Represents the final enriched planning artifacts after graph analysis and reporting.
12	Dependency Graph Definition	$(G = (V, E))$	Defines the task dependency graph, where (V) is the set of task nodes and (E) is the set of

			dependency edges.
13	Project-Level Effort Aggregation	$\left(H = \sum_{i=1}^{n} h_i\right)$	Computes the total estimated project effort by summing task-level effort values.
14	Generic Data-Driven Effort Prediction	$\left(\hat{E} = f(x_1, x_2, \dots, x_n)\right)$	Represents historical regression or machine-learning-based effort estimation from project features.
15	COCOMO II Effort Model	$\left(PM = A \times Size^{E} \times \prod EM_i\right)$	Estimates software effort in person-months based on size, scale exponent, and effort multipliers.
16	Use Case Points Formula	$(UCP = (UAW + UUCW) \times TCF \times ECF)$	Estimates software effort using actor weights, use-case weights, and adjustment factors.
17	PERT Expected Time	$\left(TE = \frac{O + 4M + P}{6}\right)$	Computes the expected duration of an activity using optimistic, most likely, and pessimistic estimates.

18	PERT Variance	$\left(\sigma^2 = \left(\frac{P-O}{6}\right)^2\right)$	Measures uncertainty in the PERT estimate of an activity duration.
19	Main Evaluation Function	$F(Y) = \frac{C_{cov}(Y) + C_{cls}(Y) + C_{cmp}(Y) + C_{dep}(Y)}{4}$	Defines the main multi-metric evaluation function used to assess overall planning quality.
20	Requirement Coverage	$Coverage = \frac{\lvert R_{covered}\rvert}{\lvert R\rvert}$	R_{covered }
21	Classification Consistency	$ClassificationConsistency = \max\left(0,\ 1 - \frac{\lvert FR_{actual} - FR_{expected}\rvert}{0.3}\right)$	FR_{actual} - FR_{expected}
22	Complexity Entropy / Complexity Calibration	$\left(C_{cmp}(Y) = \frac{-\sum_{i=1}^{k} p_i \log_2 p_i}{\log_2 5}\right)$	Evaluates the realism and diversity of complexity-level distribution across generated tasks.
23	Dependency Coherence	$\left(\ C_{dep}(Y) = \max(0,\ 1 - 0.1W)\ \right)$	Measures structural soundness of the dependency graph using warning-based penalties.
24	Overall Planning Score	$\left(Overall = \frac{Coverage + Classification + Complexity + Depe}{4}\right.$	Produces the aggregate planning-quality score used in benchmark reporting.

25	Mean Magnitude of Relative Error	$MMRE = \frac{1}{n}\sum_{i=1}^{n}\frac{\lvert A_i - \widehat{A}_i\rvert}{A_i}$	A_{i} - P_{i}
26	Prediction at 25%	$PRED(25) = \frac{1}{n}\sum_{i=1}^{n} I\left(\frac{\lvert A_i - \widehat{A}_i\rvert}{A_i} \leq 0.25\right)$	$A_{\mathrm{i}}P_{\mathrm{i}}$
27	F1 Score	$\left(F1 = \frac{2PR}{P+R}\right)$	Measures classification quality by combining precision and recall.
28	Critic Penalty Score	$\left(S_{critic}(Y) = \max\left(0,\ 1 - \sum_{j=1}^{m}\pi_j\right)\right)$	Computes the critic score after deducting penalties for structural and logical issues.
29	Risk Severity Score	$\left(S_{risk}(Y) = \min\left(1,\ \sum_{i=1}^{n}\rho_i\right)\right)$	Aggregates project risks into a normalized project-level risk score.
30	Retrieval-Based Effort Estimate	$\left(H_{rag} = \frac{\sum_{i=1}^{n} h_i w_i}{\sum_{i=1}^{n} w_i},\quad w_i = \frac{1}{\max(d_i, \epsilon)}\right)$	Computes an effort estimate from retrieved similar tasks using inverse-distance weighting.
31	Final Hybrid Effort Estimate	$\left(H_{final} = \alpha H_{rag} + (1-\alpha)H_{rule}\right)$	Produces the final effort estimate by blending retrieval-based and rule-based estimates.

Glossary of Terms

Term	Full Form / Expanded Name	Definition in This Thesis
AI	Artificial Intelligence	The broad field of computer systems designed to perform tasks that normally require human intelligence.
AI PM	AI Project Manager	The intelligent planning concept behind the proposed system; used to denote an AI-based project management and planning assistant.
AI PM	Proposed System Name	The hybrid intelligent framework proposed in this thesis for transforming software requirements into structured planning artifacts.
Software Project Planning	—	The process of converting project requirements into tasks, schedules, dependencies, estimates, and execution structures.
Automated Planning	—	The use of computational methods to generate or support planning decisions with limited human intervention.
Hybrid Planning Framework	—	A planning architecture that combines more than one computational paradigm, such as LLM reasoning, retrieval, rules, and graph analytics.
Multi-Agent System	MAS	A system composed of multiple specialized agents that cooperate to solve complex tasks.
Agent	—	A computational entity responsible for performing a specific function in the planning workflow.
Planner Agent	—	The component responsible for interpreting requirements and generating the initial task plan.
Critic Agent	—	The component responsible for validating, reviewing, and scoring the generated plan.
Monitor Agent	—	The component responsible for tracking execution progress against the generated plan.
Risk Analyzer	—	The component responsible for identifying planning risks, bottlenecks, and vulnerabilities.
Effort Estimator	—	The component responsible for assigning time and workload estimates to generated tasks.

Sprint Planner	—	The component responsible for grouping tasks into delivery-oriented sprint structures.
Local LLM	Local Large Language Model	A language model executed locally rather than through a remote cloud API.
LLM	Large Language Model	A Transformer-based model trained on large text corpora and used for reasoning, generation, and structured task decomposition.
Transformer	—	The neural network architecture based on self-attention that underlies modern large language models.
NLP	Natural Language Processing	The field concerned with enabling machines to process and interpret human language.
Requirement	—	A statement describing a needed system behavior, constraint, or quality attribute.
Requirements Engineering	RE	The discipline concerned with eliciting, analyzing, structuring, validating, and managing requirements.
Functional Requirement	FR	A requirement describing what the system must do in terms of behavior or services.
Non-Functional Requirement	NFR	A requirement describing quality constraints such as security, performance, reliability, or usability.
Requirement Classification	—	The process of categorizing requirements, especially into FR and NFR types.
Requirement Coverage	Coverage	The proportion of source requirements represented in the generated plan.
Requirement Traceability	—	The ability to link generated planning artifacts back to their original source requirements.
Requirement Normalization	—	The transformation of raw requirement text into structured and machine-processable requirement items.
Requirement Parsing	—	The extraction of explicit requirement units from free-form project descriptions or templates.
Structured Requirement Template	—	A formatted requirements representation containing labeled requirement blocks and fields.
Project Brief	—	A narrative description of the software project used as a planning input.
Brief Parser	—	The parser used to extract structured requirement items from narrative project briefs.

Template Parser	—	The parser used to extract structured requirement items from requirement templates.
Task Decomposition	—	The process of breaking high-level requirements into smaller actionable planning tasks.
Task List	—	The set of structured tasks produced by the planning system.
Planning Artifact	—	Any structured output generated by the system, such as tasks, graphs, sprint plans, summaries, or reports.
Planning Pipeline	Pipeline	The ordered sequence of processing stages through which requirements are transformed into planning outputs.
Multi-Stage Pipeline	—	A staged architecture in which the output of each phase becomes the input to the next phase.
Inference	—	The computational reasoning process used to generate classifications, tasks, dependencies, or estimates.
Structured Output Inference	—	LLM inference constrained to produce a specific machine-readable structure such as JSON.
Deterministic Validation	—	Validation based on explicit rules and checks rather than probabilistic generation.
Rule-Based System	—	A system that applies hand-crafted or programmatic rules to produce or verify outputs.
Rule-Based Fallback	—	A deterministic backup path used when LLM generation fails or is rejected.
Fallback Rate	—	The proportion of cases in which the system falls back from LLM mode to rule-based mode.
Direct Generation Rate	—	The proportion of benchmark cases completed successfully through direct LLM generation without fallback.
Retrieval-Augmented Generation	RAG	A method that augments model reasoning using externally retrieved contextual information.
Retrieval-Augmented Knowledge	—	Knowledge supplied to the system by semantic retrieval from the vector knowledge base.
Knowledge Base	KB	The collection of stored planning examples, estimation patterns, and historical records used for retrieval support.

Vector Knowledge Base	Vector KB	A knowledge base whose records are stored and searched in embedding space using semantic similarity.
Vector Database	—	A database designed to store and retrieve embedding vectors efficiently.
ChromaDB	—	The vector database used in this project to persist and retrieve semantically indexed planning knowledge.
Embedding	—	A dense numerical representation of text used for semantic comparison and retrieval.
SentenceTransformer	—	The embedding model family used to convert textual planning knowledge into vectors.
Semantic Retrieval	—	The retrieval of similar records based on meaning rather than exact keyword matching.
Similarity Search	—	The process of finding records close to a query in vector space.
Historical Records	—	Past project-like records used to support calibration and planning reference.
Planning Examples	—	Stored examples of decomposed or structured plans used to guide retrieval-supported reasoning.
Estimation Patterns	—	Reusable patterns that support the calibration of effort estimates.
COCOMO II	Constructive Cost Model II	A classical parametric model for software effort and schedule estimation.
Function Point Analysis	FPA	A functional sizing method that estimates software size from the user's perspective.
Use Case Points	UCP	An estimation method based on use-case and actor complexity.
PERT	Program Evaluation and Review Technique	A scheduling and estimation technique based on optimistic, likely, and pessimistic time estimates.
CPM	Critical Path Method	A network-based scheduling method used to identify the longest dependency-sensitive execution path.
Desharnais Regression	—	A historical regression-based software effort estimation baseline derived from project datasets.
ISBSG	International Software Benchmarking Standards Group	A well-known source of software project benchmarking data used in effort estimation research.
Benchmark	—	A controlled set of evaluation samples used to test system performance.
Benchmark Sample	—	One case within the benchmark dataset used for evaluation.

Ground Truth	—	The expected benchmark reference used to evaluate the generated outputs.
Constraint-Based Ground Truth	—	A benchmark design in which expected outputs are defined through structural constraints rather than exact text matching.
Ablation Study	—	An experimental comparison used to measure the contribution of different system components or configurations.
Evaluation Mode	—	A specific operational configuration such as `rules`, `rules_kb`, `llm`, `llm_kb`, or `zero_shot`.
Zero-Shot	—	A configuration in which the model performs a task without task-specific examples in the prompt.
Prompt Engineering	—	The design of instructions and context used to guide model behavior.
Prompt	—	The input instruction provided to the language model.
JSON	JavaScript Object Notation	The structured machine-readable format used for task and planning outputs.
Schema	—	The formal structure that defines the required fields and format of an output object.
Schema Validation	—	The process of checking whether an output conforms to the expected structure.
Complexity Level	—	The assigned difficulty rating of a task, typically on a discrete multi-level scale.
Complexity Entropy	—	A normalized Shannon-entropy measure used to evaluate how realistically complexity levels are distributed across tasks.
Shannon Entropy	—	A mathematical measure of uncertainty or diversity used here to assess complexity distribution.
FR/NFR Ratio	—	The balance between functional and non-functional tasks or requirements in a plan.
Classification Consistency	—	A score measuring how closely the generated FR/NFR balance matches the expected distribution.
Critic Score	—	The plan-quality score produced by the Critic Agent after issue-based penalty calculation.
Penalty Scheme	—	The rule set used to deduct points from a score according to error severity.
Error	—	A serious structural or logical issue that can cause plan rejection.

Warning	—	A moderate issue that lowers confidence but does not necessarily invalidate the plan.
Informational Issue	Info	A minor issue recorded for transparency and quality review.
Approved	—	A critic decision state indicating that the generated plan is acceptable.
Needs Revision	—	A critic decision state indicating that the plan is usable but requires improvement.
Rejected	—	A critic decision state indicating that the plan failed validation and should not proceed.
Dependency	—	A relationship showing that one task must be completed before another can begin.
Dependency Graph	—	The graph representation of tasks and their precedence relations.
Graph	—	A mathematical structure composed of nodes and edges used to model relationships.
Node	—	A graph element representing a task in the dependency model.
Edge	—	A graph relation representing a dependency between two tasks.
Directed Graph	—	A graph in which edges have direction, indicating precedence.
DAG	Directed Acyclic Graph	A directed graph with no cycles, used to represent valid execution dependencies.
Cycle	—	A circular dependency pattern that makes execution ordering invalid.
Dependency Coherence	—	A structural metric measuring the validity and plausibility of the generated dependency graph.
Root Task	—	A task with no predecessors in the dependency graph.
Leaf Task	—	A task with no successors in the dependency graph.
Bottleneck Task	—	A task whose position or connectivity makes it critical to execution flow.
Parallel Task Group	—	A set of tasks that can be executed concurrently because they do not depend on one another.
Critical Path	—	The longest dependency-sensitive execution path that determines schedule sensitivity.
Critical Path Analysis	—	The process of identifying the most execution-sensitive dependency chain in the plan.

Graph Analytics	—	Analytical operations performed on the dependency graph to derive execution insights.
Graph-Based Modeling	—	The use of graph structures to represent planning relationships and execution constraints.
GNN	Graph Neural Network	A neural model designed to learn from graph-structured data; discussed as a related modern technique.
NetworkX	—	The Python library used to construct and analyze dependency graphs in the project.
Sprint	—	A time-bounded delivery iteration grouping related tasks.
Sprint Plan	—	The delivery-oriented grouping of tasks into executable sprints.
Team Allocation	—	The distribution of tasks or roles across team members or skill categories.
Owner Role	—	The suggested functional role responsible for a task, such as frontend or backend developer.
Skill Requirement	—	The technical competence needed to complete a given task.
Effort Estimation	—	The process of predicting the amount of work needed to complete a task or project.
Estimated Hours	—	The predicted labor time assigned to a task in hours.
Estimated Days	—	The predicted labor time assigned to a task in days.
Team Size Suggestion	—	The recommended number of contributors for a task or sprint.
Effort Calibration	—	The adjustment of base estimates using retrieved patterns or reference knowledge.
MMRE	Mean Magnitude of Relative Error	A standard metric used to evaluate effort-estimation accuracy.
PRED(25)	Prediction at 25%	The proportion of estimates whose relative error does not exceed 25%.
F1 Score	—	The harmonic mean of precision and recall, used here for task-type classification evaluation.
F1-FR	F1 Score for Functional Requirements	The F1 score computed for the functional class.
F1-NFR	F1 Score for Non-Functional Requirements	The F1 score computed for the non-functional class.

Precision	—	The proportion of predicted positives that are correct.
Recall	—	The proportion of actual positives that are correctly identified.
Overall Score	—	The aggregate normalized planning-quality score combining coverage, classification, complexity, and dependency metrics.
Average	Mean	The arithmetic mean of a metric across benchmark samples.
Standard Deviation	SD	A statistical measure of variation around the mean.
Success Rate	—	The percentage of benchmark runs that complete successfully.
Pass Rate	—	The percentage of benchmark samples meeting the required acceptance criteria.
Structural Evaluation	—	Evaluation focused on coverage, dependency validity, and critic-based plan acceptability.
Quantitative Evaluation	—	Evaluation focused on numerical performance metrics such as MMRE, PRED(25), and F1 scores.
Multi-Metric Evaluation	—	An evaluation strategy that combines several metrics rather than relying on a single score.
Experimental Protocol	—	The defined procedure followed during benchmark execution and evaluation.
Experimental Environment	—	The software and runtime context used to execute the system and collect results.
Local-First Architecture	—	A design philosophy in which computation and storage are primarily executed locally.
Deployability	—	The degree to which the system can be practically run in real environments.
Transparency	—	The property of making system behavior understandable and inspectable.
Interpretability	—	The ease with which a human can understand how the system reached an output.
Trustworthiness	—	The degree to which the system can be relied upon to produce valid and reviewable outputs.
Robustness	—	The ability of the system to maintain acceptable performance under failure or variation.

Reliability	—	The consistency with which the system produces correct or acceptable outputs.
Scalability	—	The ability of the system to handle increasing complexity, data, or usage demand.
Reproducibility	—	The ability to repeat the same experiment and obtain comparable results.
Validation	—	The process of checking whether the generated plan satisfies structural and logical requirements.
Verification	—	The process of confirming that the produced artifacts conform to expected formal conditions.
Risk Register	—	A structured artifact listing identified risks, their severity, and mitigation suggestions.
Risk Score	—	A numeric summary of the aggregate severity of planning risks.
Risk Mitigation	—	An action or strategy intended to reduce planning-related risks.
Operational Reliability	—	The practical reliability of the system during actual runs, including fallback behavior and acceptance rates.
Dashboard	—	The main visual interface used to inspect generated project plans and summaries.
Graph View	—	The visual interface used to inspect the task dependency graph.
Evaluation Page	—	The visual interface used to inspect benchmark metrics and assessment results.
API	Application Programming Interface	The interface through which the backend services expose planning and evaluation functionality.
FastAPI	—	The Python web framework used to implement the backend API.
Backend	—	The server-side part of the system responsible for planning logic, evaluation, and data handling.
Frontend	—	The client-side interface responsible for presenting system outputs visually.
React	—	The frontend library used to build the web-based user interface.
Vite	—	The frontend build and development tool used in the project.
Pydantic	—	The Python library used for data modeling and schema validation.

Ollama	—	The local model-serving environment used to host and call the local LLM.
Qwen	—	The LLM family used as the local reasoning model in the project environment.
Sentence Embedding Model	—	The model used to map text into vectors for semantic retrieval.
Committee Brief	—	A high-level summary artifact prepared for presentation or academic review.
Plan Summary	—	A summarized artifact describing the generated project plan at a high level.
Reviewable Artifact	—	An output designed to be inspected and assessed by human evaluators.
Related Work	—	The set of previous studies reviewed to position the thesis within the literature.
Research Gap	—	The unresolved problem or limitation in prior work that motivates the proposed study.
Proposed Architecture	—	The system design advanced by this thesis to solve the identified planning problem.
Research Objective	—	A declared goal that the thesis seeks to achieve through design, implementation, and evaluation.
Research Contribution	—	The original value added by the thesis to knowledge, methodology, or implementation.
Theoretical Foundation	—	The conceptual and scientific basis upon which the proposed system is built.
Methodology	—	The structured approach used to design, implement, and evaluate the proposed framework.
Case Study	—	A specific example or scenario used in related work or evaluation to illustrate system behavior.
Benchmark Dataset	—	The collection of benchmark cases used for controlled experimental assessment.
Sample Brief	—	A representative project description used as an evaluation input.
Software Engineering	SE	The discipline concerned with the systematic development and management of software systems.
Human-in-the-Loop	—	An approach in which human review or intervention is incorporated into an AI-supported system.
Planning Under Uncertainty	—	A planning setting in which future conditions, effort, or outcomes are not fully known in advance.

Causal Reasoning	—	A reasoning approach that models cause-and-effect relations to support decision making.
Quality Attribute	—	A non-functional property such as security, performance, maintainability, or usability.
KAOS	Knowledge Acquisition in Automated Specification	A goal-oriented requirements engineering modeling approach mentioned in the reviewed literature.

1 Chapter one: introduction

1.1 General Overview

The contemporary software industry is increasingly characterized by accelerated delivery cycles, growing system complexity, multidimensional stakeholder requirements, and heightened expectations for reliability, scalability, and maintainability. In this environment, the early planning phase of software projects has become a critical determinant of project success, as errors or ambiguities introduced at the requirements and planning stage frequently propagate into downstream development, testing, deployment, and maintenance activities [1]. Despite the strategic importance of this phase, many software teams still rely on manual planning practices, informal interpretation of requirements, and experience-driven decomposition of work, which often lead to inconsistent task granularity, weak traceability, inaccurate effort estimation, and poorly structured execution plans [2].

The challenge becomes more pronounced when project requirements are expressed in natural language, as is common in academic, startup, and small-team environments. Raw requirement documents often contain ambiguity, mixed levels of abstraction, overlapping functional and non-functional concerns, and implicit dependencies that are difficult to identify through conventional manual analysis alone [3]. As a result, transforming textual project briefs into actionable project management artifacts, such as task breakdowns, dependency graphs, sprint plans, and risk assessments, remains a labor-intensive and error-prone process. Traditional estimation approaches, including expert judgment, rule-of-thumb decomposition, and classical sizing models, can provide partial support; however, they typically do not deliver an integrated, automated, and traceable planning workflow from unstructured requirements to executable planning outputs [4].

Recent advances in artificial intelligence, particularly Large Language Models (LLMs), retrieval-augmented generation, and agent-oriented reasoning systems, have introduced new opportunities for addressing these limitations [5]. Unlike static rule-based tools or isolated estimation calculators, modern AI-driven planning systems can analyze natural language requirements, infer latent structure, decompose complex problem statements, and generate context-aware planning artifacts at a level of abstraction closer to human project reasoning. Nevertheless, the practical application of such models in project planning remains constrained by several concerns, including hallucination, inconsistency in structured outputs, lack of explainability, sensitivity to prompt design, and dependence on cloud-based infrastructure [6]. These limitations make it necessary to design planning frameworks that combine the reasoning capabilities of LLMs with deterministic validation, domain knowledge grounding, and auditable fallback mechanisms.

Within this context, the present project proposes AI PM, an intelligent AI-driven project planning framework designed to convert software requirement documents into structured project management artifacts through a hybrid, local-first architecture. The system combines LLM-guided planning with rule-based validation, retrieval from a curated knowledge base, effort estimation, dependency analysis, sprint planning, and risk

assessment. Rather than depending exclusively on generative AI, the framework adopts a hybrid reasoning strategy in which the language model acts as a planning reasoner, while deterministic validators ensure structural correctness, requirement traceability, classification consistency, and graph integrity before any generated plan is accepted. This design enables the system to preserve the adaptability of AI while maintaining the reliability, reproducibility, and transparency expected in academic and professional planning environments [7].

AI PM extends beyond simple task generation by producing a comprehensive planning pipeline that begins with parsing narrative or template-based requirement inputs and proceeds toward the generation of task lists, planning summaries, dependency-aware execution structures, and analytical reports. The framework employs a local LLM deployment for privacy-preserving reasoning, a vector-based knowledge base for contextual retrieval, and graph-based analysis techniques to model task interdependencies and identify critical execution paths. In addition, the system incorporates critic-style validation, risk analysis, and sprint-oriented organization to transform raw requirements into artifacts that are not only machine-readable, but also practically useful for project managers, student teams, and software engineering researchers [8].

As intelligent planning systems become more prominent, issues such as output quality, interpretability, computational efficiency, and robustness under imperfect input conditions have emerged as central research concerns. Many existing AI-assisted software planning solutions remain limited either by overreliance on opaque generative behavior, insufficient validation of generated outputs, weak integration with estimation knowledge, or inadequate evaluation against reproducible benchmarks [9]. In response, this study contributes a hybrid intelligent planning framework that emphasizes structured generation, knowledge-grounded estimation, local deployment, and measurable evaluation. By integrating LLM reasoning, retrieval-augmented support, deterministic safeguards, and graph-based planning analytics, the proposed system aims to establish a strong foundation for the next generation of intelligent software project management systems.

1.2 Problem Statement and Significance

Contemporary software project planning practices continue to suffer from substantial structural limitations that reduce their effectiveness in transforming raw requirements into reliable execution plans. In many academic, startup, and small-team development environments, project planning remains heavily dependent on manual interpretation of requirement documents, informal decomposition of work, and experience-driven estimation strategies. Although such approaches may be workable for small and well-defined tasks, they become increasingly inadequate as project scope expands, requirements diversify, and functional and non-functional constraints become more intertwined. As a result, planning outputs often exhibit inconsistent task granularity, weak traceability to source requirements, incomplete dependency modeling, and unreliable effort estimation, all of which negatively affect delivery predictability and project control.

A major dimension of this problem lies in the nature of software requirements themselves. Requirements are commonly expressed in natural language, which is inherently prone to ambiguity, incompleteness, overlap, and variation in abstraction level. In practice, a single project brief may combine business goals, user-facing features, technical constraints, quality requirements, security expectations, and optional enhancements within the same textual artifact. Traditional planning workflows rarely provide a systematic and reproducible mechanism for extracting this information, classifying it accurately, and converting it into coherent project management artifacts. Consequently, important requirements may be overlooked, non-functional constraints may be underrepresented, and dependency structures may be constructed in a superficial or ad hoc manner, thereby increasing the risk of downstream rework, estimation errors, and execution bottlenecks.

Another critical challenge concerns the lack of integration between planning, validation, and evaluation. Existing manual and semi-automated planning approaches often generate task lists or effort estimates in isolation, without sufficient mechanisms to verify structural correctness, detect internal inconsistencies, or evaluate the quality of the produced plan against measurable criteria. This limitation is particularly significant in research and educational contexts, where planning outputs must not only be useful in practice, but also transparent, reproducible, and assessable. Without systematic validation and evaluation, planning decisions remain difficult to audit, compare, or improve, thereby weakening both practical reliability and scientific rigor.

Recent advances in artificial intelligence, especially Large Language Models, offer promising capabilities for interpreting natural language requirements and generating structured outputs. However, relying exclusively on generative AI introduces additional risks. LLMs may hallucinate tasks, omit critical constraints, produce invalid structures, or generate plans that appear plausible while lacking internal consistency and traceability. These weaknesses are especially problematic in project planning, where inaccurate decomposition or misleading dependency relationships can affect implementation sequencing, resource allocation, risk

exposure, and stakeholder expectations. Therefore, the central problem is not merely how to apply AI to software planning, but how to design an intelligent planning framework that preserves the expressive and reasoning strengths of LLMs while addressing their reliability limitations through deterministic safeguards, contextual grounding, and measurable quality control.

The core research problem addressed in this study is the development of a practical and trustworthy system capable of converting natural-language software requirements into structured, dependency-aware, and evaluation-ready project planning artifacts under realistic operational constraints. These constraints include limited computational infrastructure, the need for local or low-cost deployment, imperfect or ambiguous input specifications, and the absence of large, domain-specific training pipelines. The significance of this research lies in its attempt to bridge the gap between contemporary AI capabilities and the operational demands of real-world software planning by introducing a hybrid approach that integrates LLM reasoning, retrieval-augmented contextual support, rule-based validation, effort estimation, dependency analysis, and risk-aware planning within a single coherent pipeline.

The proposed AI PM framework addresses this gap by offering an intelligent project planning model that moves beyond isolated task generation toward a comprehensive planning process. It transforms raw project briefs into validated task structures, planning summaries, dependency graphs, sprint-oriented artifacts, and analytical outputs that can support both practical execution and academic evaluation. In doing so, the project contributes not only a software prototype, but also a methodological perspective on how hybrid AI systems can be engineered to produce planning outputs that are more reliable, interpretable, and reproducible than purely manual or purely generative alternatives. This makes the research significant for software engineering practice, AI-assisted project management, and the broader study of intelligent systems for requirements-driven planning.

1.3 Project Objectives and Scope of Work

Building upon the aforementioned research problem, this project aims to design and implement an AI-driven intelligent academic guidance system that overcomes the limitations of traditional data analysis to deliver practical solutions deployable in real-world educational environments. The research focuses on transforming raw academic data into precise, reliable recommendations that support decision-making at both the student and institutional levels.

1.3.1 Project Objectives

The project seeks to achieve the following interconnected objectives:

1.Development of a Comprehensive Intelligent Academic Guidance System

To create an AI-driven system capable of inferring precise academic recommendations and analytical insights from data, thereby enabling highly personalized guidance tailored to individual student profiles.

2.Analysis and Comparison of AI Architectures

To investigate the performance of various architectural approaches, including Retrieval-Augmented Generation (RAG) and Agentic AI, analyzing the strengths and limitations of each within diverse academic contexts.

3.Quantitative and Qualitative Performance Evaluation

To measure recommendation effectiveness using quantitative metrics such as response accuracy, user satisfaction rates, and predictive model efficacy, alongside qualitative assessment focusing on practical feasibility and alignment with actual academic requirements.

4. Achievement of Practical Balance Between Accuracy and Efficiency

To optimize system efficiency regarding computational resource consumption and response latency, ensuring deploy ability in resource-constrained environments without compromising output quality.

1.3.2 Scope of Work (Execution Framework)

To achieve the aforementioned objectives, the project adopts a structured work framework encompassing the following components:

•Data Preparation and Processing Collection and processing of extensive academic datasets comprising student records distributed across five distinct majors within the Faculty of Informatics Engineering. This involves implementing specialized import functions for processing CSV and ZIP files, extracting relevant academic data, and organizing it within a structured database schema.

•Pipeline Design and Construction Development of an integrated processing pipeline encompassing data preprocessing, intelligent analysis, and results evaluation through an Edge Functions architecture comprising more than 35 specialized functions. This includes a hybrid RAG system for knowledge retrieval and an intelligent chat system with an orchestration layer for workflow management.

•Experimental Implementation and Output Analysis Development and testing of selected systems using the prepared datasets, with systematic analysis of quantitative and visual outputs through multiple subsystems including academic performance analysis, academic decision simulation, early warning systems, career pathway analysis, and proactive alert

systems. This component utilizes Large Language Model APIs (LLM APIs) for generating intelligent recommendations and analyses.

•Academic Documentation Comprehensive documentation of all project phases, beginning with theoretical and methodological frameworks, proceeding through experimental procedures, and concluding with results presentation, discussion, and future recommendation

1.4 Research Contributions

This study contributes to the domain of intelligent software project planning through a set of technically grounded, experimentally oriented, and practically deployable advancements, which may be summarized as follows:

Hybrid Local-First Intelligent Planning Architecture

The project presents a hybrid planning architecture that combines local Large Language Model reasoning, retrieval-augmented contextual support, and deterministic rule-based validation within a single coherent framework. Rather than treating generative output as inherently trustworthy, the system positions the LLM as a planning reasoner whose output is continuously constrained, validated, and, when necessary, replaced through a structured fallback mechanism. This design contributes a practical model for deploying AI-assisted planning in privacy-sensitive and resource-constrained environments without exclusive dependence on cloud-based inference services.

Knowledge-Grounded Requirement-to-Plan Transformation Pipeline

The study introduces a complete transformation pipeline that converts raw software requirements, expressed either as narrative project briefs or structured templates, into normalized planning artifacts. This pipeline integrates requirement parsing, task decomposition, functional and non-functional classification, effort estimation, dependency extraction, sprint-oriented organization, and risk-aware summarization. The contribution lies not merely in task generation, but in demonstrating how knowledge-grounded AI can be embedded into a traceable planning workflow that preserves linkage between source requirements and downstream execution structures.

Reliability-Oriented Integration of LLM Reasoning with Deterministic Safeguards

A major contribution of the project is the explicit integration of LLM-based planning with critic-style validation, structural verification, and graph integrity checks. This ensures that

generated plans satisfy essential quality requirements such as valid task structure, consistent dependency relationships, bounded complexity levels, and interpretable requirement coverage. By incorporating deterministic safeguards at the acceptance stage, the system addresses one of the central weaknesses of purely generative planning approaches: the production of plausible yet structurally unreliable outputs.

Contextual Knowledge Base for Planning and Estimation Support

The project contributes a retrieval-supported planning mechanism in which a curated knowledge base of planning examples, task-level estimation patterns, and historical calibration records is used to enrich both decomposition and estimation stages. This demonstrates how retrieval-augmented planning can improve consistency and reduce unsupported generation by grounding the reasoning process in reusable domain knowledge. The contribution is particularly significant in showing that AI-assisted software planning can benefit from contextual memory structures without requiring custom model training.

Integrated Multi-Artifact Planning Framework

Unlike approaches that terminate at a flat task list, the proposed system generates a broader family of project management artifacts, including validated tasks, planning summaries, dependency graphs, sprint plans, risk reports, monitoring outputs, and export-ready deliverables. This transforms the system from a narrow decomposition tool into a more comprehensive intelligent planning environment. The contribution here lies in the unification of multiple planning functions within a single framework capable of supporting both analytical evaluation and practical project coordination.

Reproducible Evaluation and Ablation-Based Validation Framework

The study further contributes a structured empirical evaluation framework for assessing intelligent project planning quality. The system is evaluated through benchmark-driven experimentation and ablation analysis across alternative planning modes, including rule-based, knowledge-assisted, and LLM-guided configurations. Evaluation criteria extend beyond basic usability to include requirement coverage, classification behavior, complexity calibration, dependency coherence, and estimation quality. This provides a reproducible basis for comparing hybrid planning strategies and strengthens the scientific validity of the proposed framework.

Practical Human-in-the-Loop Planning Support

The project also contributes an applied interaction layer through which generated plans can be reviewed, interpreted, communicated, and refined by human users. By incorporating review, feedback, explanation, and monitoring interfaces, the system acknowledges that intelligent planning in real settings should augment human decision-making rather than replace it. This contribution is important because it situates the framework within an

operational context where trust, interpretability, and controlled supervision are essential for adoption.1.4.3 Research Contributions

This study contributes to the domain of intelligent software project planning through a set of technically grounded, experimentally oriented, and practically deployable advancements, which may be summarized as follows:

Hybrid Local-First Intelligent Planning Architecture

The project presents a hybrid planning architecture that combines local Large Language Model reasoning, retrieval-augmented contextual support, and deterministic rule-based validation within a single coherent framework. Rather than treating generative output as inherently trustworthy, the system positions the LLM as a planning reasoner whose output is continuously constrained, validated, and, when necessary, replaced through a structured fallback mechanism. This design contributes a practical model for deploying AI-assisted planning in privacy-sensitive and resource-constrained environments without exclusive dependence on cloud-based inference services.

Knowledge-Grounded Requirement-to-Plan Transformation Pipeline

The study introduces a complete transformation pipeline that converts raw software requirements, expressed either as narrative project briefs or structured templates, into normalized planning artifacts. This pipeline integrates requirement parsing, task decomposition, functional and non-functional classification, effort estimation, dependency extraction, sprint-oriented organization, and risk-aware summarization. The contribution lies not merely in task generation, but in demonstrating how knowledge-grounded AI can be embedded into a traceable planning workflow that preserves linkage between source requirements and downstream execution structures.

Reliability-Oriented Integration of LLM Reasoning with Deterministic Safeguards

A major contribution of the project is the explicit integration of LLM-based planning with critic-style validation, structural verification, and graph integrity checks. This ensures that generated plans satisfy essential quality requirements such as valid task structure, consistent dependency relationships, bounded complexity levels, and interpretable requirement coverage. By incorporating deterministic safeguards at the acceptance stage, the system addresses one of the central weaknesses of purely generative planning approaches: the production of plausible yet structurally unreliable outputs.

Contextual Knowledge Base for Planning and Estimation Support

The project contributes a retrieval-supported planning mechanism in which a curated knowledge base of planning examples, task-level estimation patterns, and historical calibration records is used to enrich both decomposition and estimation stages. This

demonstrates how retrieval-augmented planning can improve consistency and reduce unsupported generation by grounding the reasoning process in reusable domain knowledge. The contribution is particularly significant in showing that AI-assisted software planning can benefit from contextual memory structures without requiring custom model training.

Integrated Multi-Artifact Planning Framework

Unlike approaches that terminate at a flat task list, the proposed system generates a broader family of project management artifacts, including validated tasks, planning summaries, dependency graphs, sprint plans, risk reports, monitoring outputs, and export-ready deliverables. This transforms the system from a narrow decomposition tool into a more comprehensive intelligent planning environment. The contribution here lies in the unification of multiple planning functions within a single framework capable of supporting both analytical evaluation and practical project coordination.

Reproducible Evaluation and Ablation-Based Validation Framework

The study further contributes a structured empirical evaluation framework for assessing intelligent project planning quality. The system is evaluated through benchmark-driven experimentation and ablation analysis across alternative planning modes, including rule-based, knowledge-assisted, and LLM-guided configurations. Evaluation criteria extend beyond basic usability to include requirement coverage, classification behavior, complexity calibration, dependency coherence, and estimation quality. This provides a reproducible basis for comparing hybrid planning strategies and strengthens the scientific validity of the proposed framework.

Practical Human-in-the-Loop Planning Support

The project also contributes an applied interaction layer through which generated plans can be reviewed, interpreted, communicated, and refined by human users. By incorporating review, feedback, explanation, and monitoring interfaces, the system acknowledges that intelligent planning in real settings should augment human decision-making rather than replace it. This contribution is important because it situates the framework within an operational context where trust, interpretability, and controlled supervision are essential for adoption.

1.5 Research Beneficiaries

The outcomes of this research are expected to provide value to several beneficiary groups across academic, technical, and organizational contexts. Because the proposed system addresses the transformation of raw software requirements into structured project planning artifacts, its relevance extends beyond a single user category and supports both practical project execution and academic investigation.

Software Engineering Students and Capstone Teams

One of the primary beneficiary groups comprises software engineering students, particularly those engaged in graduation projects, capstone courses, and team-based development activities. Such users frequently face difficulties in translating initial project ideas and textual requirement statements into coherent task structures, realistic effort estimates, and executable delivery plans. The proposed framework offers these users an intelligent planning environment capable of improving requirement interpretation, clarifying implementation priorities, and reducing the uncertainty commonly associated with early project organization.

Academic Supervisors and Project Evaluators

The system can also support supervisors, instructors, and academic evaluators who oversee student software projects. By generating structured planning artifacts such as validated task lists, dependency graphs, sprint plans, and risk summaries, the framework provides a clearer basis for assessing project scope, feasibility, planning quality, and execution readiness. This contributes to more objective academic review and facilitates evidence-based supervision during project development lifecycles.

Project Managers and Software Team Leads

In professional and semi-professional software environments, project managers and team leads may benefit from the framework as a decision-support tool during the planning phase. The system assists in decomposing requirement documents, identifying dependency relationships, estimating work effort, and highlighting planning risks before implementation begins. This can enhance planning consistency, improve resource allocation, and reduce the likelihood of downstream delays caused by weak early-stage analysis.

Requirements Engineers and Business Analysts

The research is also relevant to professionals concerned with requirements engineering and systems analysis. Since the proposed system processes narrative and structured requirements and converts them into traceable planning outputs, it offers a practical example of how artificial intelligence can support the interpretation, normalization, and operationalization of requirement artifacts. This may help analysts improve traceability between stakeholder needs and technical planning decisions.

Researchers in Artificial Intelligence and Software Engineering

From a scientific perspective, the study provides value to researchers working at the intersection of artificial intelligence, software engineering, requirements engineering, and intelligent decision-support systems. The hybrid design of the framework, which combines LLM reasoning, retrieval-augmented support, deterministic validation, and empirical evaluation, offers a reproducible case study for investigating trustworthy AI-assisted planning under realistic engineering constraints.

Startups and Resource-Constrained Development Environments

Emerging software teams, small development groups, and startup environments may particularly benefit from the system's local-first and hybrid operational model. Such contexts often lack formal planning infrastructure, large teams, or extensive project management resources. By automating key planning functions while preserving transparency and controllability, the framework provides a practical solution for organizations seeking to improve planning quality without incurring the cost of heavyweight enterprise tooling.

Overall, the significance of the proposed research lies in its ability to serve both educational and professional communities by delivering an intelligent planning framework that enhances requirement understanding, strengthens execution readiness, and supports more structured and reliable software project management practices.

2 Chapter Two: Theoretical Foundations

2.1 Introduction

This chapter presents the mathematical and computational foundations underlying the proposed **AI PM** framework. The system is not based on a single algorithmic paradigm; rather, it is built upon an integrated set of theoretical principles drawn from natural language processing, information retrieval, large language model reasoning, rule-based validation, graph theory, effort estimation, and evaluation science. Together, these foundations enable the transformation of raw software requirements into structured, validated, and execution-oriented project planning artifacts.

2.2 Natural Language Requirements Processing

A fundamental premise of the proposed system is that software requirements are commonly expressed in natural language, which is inherently ambiguous, variable in structure, and mixed in abstraction level. Consequently, the first theoretical foundation of the project lies in **natural language requirements processing**, where textual input is converted into normalized requirement units suitable for downstream computation.

From a computational perspective, this process involves segmentation, normalization, clause extraction, and semantic interpretation. A raw project brief is treated as an unstructured text sequence (X), which is transformed into a set of requirement items:

$$\mathbf{R} = \boldsymbol{r_1, r_2, r_3, \ldots\ldots, r_n}$$

where each (r_i) represents an atomic or semi-atomic requirement statement. This transformation is essential because planning algorithms operate more effectively on normalized requirement units than on free-form paragraphs. The purpose of this stage is therefore to reduce ambiguity, preserve traceability, and establish a formal bridge between textual input and structured planning logic.

2.3 Structured Representation of Planning Artifacts

The proposed framework depends on the principle that planning outputs must be represented as structured, machine-interpretable objects rather than informal textual descriptions. Each generated task can be modeled as a structured tuple:

$$t_{i=}(id_i, title_i, type_i, c_i, d_i, h_i, dep_i, s_i)$$

where:

- id_i denotes the task identifier
- $title_i$ denotes the task title
- $type_i$ denotes the requirement class
- c_i denotes complexity
- d_i denotes description
- h_i denotes effort
- dep_i denotes dependencies
- s_i denotes source requirement reference.

This structured representation enables validation, serialization, graph construction, and analysis.

2.4 Retrieval-Augmented Reasoning and Vector Similarity

A core theoretical component of AI PM is retrieval-augmented generation (RAG), which improves reasoning quality by grounding the system in previously stored planning knowledge. Instead of generating tasks purely from model memory, the framework retrieves semantically relevant examples and estimation patterns from a vector-based knowledge base.

Each textual item is encoded into an embedding vector in a high-dimensional semantic space:

$$e(x) \in \mathbb{R}^{\mathrm{d}}$$

. Similarity is computed using cosine similarity:

$$sim(a, b) = \frac{a \cdot b}{|a||b|}$$

This formulation allows the system to retrieve contextually related planning records even when exact wording differs. The theoretical significance of this mechanism lies in reducing unsupported generation, improving contextual consistency, and enabling knowledge-grounded decomposition and estimation without retraining the underlying language mode.

2.5 Large Language Model Reasoning

The generative planning component of the system is founded on the theory of probabilistic sequence modeling used by large language models. Given an input prompt x, the model produces an output sequence y by estimating:

$$P(y \mid x)$$

In the case of AI PM, the prompt includes normalized requirements, retrieved knowledge base context, and output constraints. The LLM therefore acts as a reasoning engine that proposes task decompositions, classifications, and dependency patterns conditioned on both input requirements and external context.

However, the project does not treat model output as inherently correct. The theoretical position adopted here is that LLM reasoning is powerful but non-deterministic; therefore, its role must be embedded within a constrained acceptance framework. This transforms the model from a sole decision-maker into one component within a broader hybrid reasoning architecture.

2.6 Rule-Based Validation and Constraint Enforcement

To ensure output reliability, the project relies on **rule-based validation**, which may be interpreted as a constraint satisfaction layer imposed over generated plans. Let Y denote the candidate task plan produced by the system. The plan is accepted only if it satisfies a set of structural and semantic constraints:

$$\forall r_j \in \mathcal{C},\ r_j(Y) = True$$

where $(\mathcal{C})$ is the set of validation constraints. These constraints include valid schema structure, acceptable complexity bounds, consistent task identifiers, absence of self-dependencies, valid requirement labels, and graph integrity conditions.

This layer is theoretically important because it introduces determinism into an otherwise probabilistic generation process. In effect, the system follows a generate-then-verify paradigm, where creativity and flexibility are provided by the language model, while correctness and consistency are enforced through explicit computational rules.

2.7 Graph Theory for Dependency Modeling

Once tasks are generated, the project models their execution structure using directed graph theory. The task plan is represented as a directed graph:

$$\boldsymbol{G = (V, E)}$$

where:

V is the set of tasks,

E is the set of directed dependency edges.

An edge (u, v) indicates that task (u) must precede task (v). For a valid project plan, the graph should form a Directed Acyclic Graph (DAG), since cyclic dependencies imply logically impossible execution orderings.

Graph-theoretic analysis enables the system to identify:

root tasks with no predecessors,

leaf tasks with no dependents,

bottleneck nodes,

parallelizable groups,

critical execution sequences.

The critical path may be conceptualized as the longest dependency-aware path in the graph under task weights:

$$\boldsymbol{CP = \mathrm{arg\,max}_{p \in \mathcal{P}} \sum_{v \in p} w(v)}$$

Where $(w(v))$ may represent estimated effort or complexity. This provides a principled basis for planning priority, schedule awareness, and risk concentration analysis.

2.8 Effort Estimation Principles

Another theoretical pillar of the system is **software effort estimation**. Rather than relying exclusively on classical parametric models or purely learned regression, AI PM adopts a hybrid estimation approach informed by requirement semantics, task structure, and retrieved knowledge patterns.

If each task (t_i) is assigned an estimated effort (h_i), the total project effort is computed as:

$$\boldsymbol{H = \sum_{i=1}^{n} h_i}$$

This aggregate effort can then be transformed into estimated days, sprint loads, or role-based allocations. The theoretical contribution here is not the use of a single universal estimation equation, but the use of contextual estimation grounded in planning examples and bounded by explicit task properties such as complexity, type, and implementation scope.

2.9 Risk-Oriented Planning Analysis

AI PM also depends on analytical principles for risk scoring and planning vulnerability assessment. The system interprets planning risk as an emergent property influenced by factors such as bottlenecks, large critical-path tasks, excessive non-functional load, structural warnings, and estimation pressure.

In abstract form, project risk can be modeled as a weighted function:

$$\boldsymbol{Risk = f(b, cp, c, q, s)}$$

where:

- b denotes bottleneck severity,
- Cp denotes critical path concentration,
- c denotes task complexity burden,
- q denotes plan quality issues,
- s denotes schedule or effort pressure.

This formulation supports the notion that project risk is not random, but inferable from structural features of the plan itself. Accordingly, risk analysis becomes a computational extension of planning rather than a separate managerial intuition.

2.10 Evaluation Theory and Performance Metrics

Requirement coverage:

$$Coverage = \frac{\text{covered requirements}}{\text{total requirements}}$$

Classification Quality, which evaluates how well the generated plan preserves expected functional and non-functional balance.

Complexity Calibration, which assesses whether the plan avoids unrealistic uniformity in task complexity.

Dependency Coherence, which measures structural soundness of the dependency graph.

Overall Planning Score, computed as an aggregate of the major normalized evaluation dimensions.

This evaluation logic is theoretically significant because it treats planning as a measurable computational output rather than a purely subjective artifact. It also enables ablation analysis to compare rule-based, retrieval-assisted, and LLM-guided configurations in a reproducible manner.

2.11 Chapter Summary

In summary, the proposed AI PM framework is grounded in a combination of computational theories that collectively support intelligent software project planning. Natural language processing enables requirement interpretation, structured representation enables machine validation, retrieval-augmented reasoning provides contextual grounding, large language models contribute generative planning capability, rule-based constraints ensure reliability, graph theory supports dependency modeling, estimation theory informs workload analysis, and evaluation metrics provide empirical validation. These theoretical foundations establish the scientific basis upon which the proposed system is designed, implemented, and assessed.

3 Chapter Three: Reference Study

3.1 Introduction

This chapter reviews the most relevant recent studies that inform the design of the proposed **AI PM** framework. The selected works were published between **2024 and 2026**, thereby satisfying the recency requirement for contemporary AI-assisted software engineering research. The review focuses on studies that are closely aligned with the core dimensions of this project: **LLM-based multi-agent orchestration, requirements structuring, planning under uncertainty, software effort estimation, role-based collaboration, and verification-aware reasoning**.

To keep the review analytically useful rather than merely descriptive, each study is examined in terms of: **(1) the research group and year, (2) the problem addressed, (3) the adopted model or framework, (4) the evaluation criteria, (5) the principal findings, and (6) the major strengths and limitations**.

The strengths and weaknesses reported below are not copied verbatim from the papers; rather, they are **critical syntheses derived from the methods and results reported in each study**.

3.2 Previous Studies in Related Domains

3.2.1 Study [4]

In one of the most influential recent studies in this area, a comprehensive work was published in **ACM TOSEM (2025)** [4]. The study did not propose a single executable tool; instead, it provided a **systematic literature review and conceptual roadmap** for the use of LLM-based multi-agent systems across the software development lifecycle. The paper analyzed how agent specialization, autonomous decomposition, and agent collaboration can improve complex software engineering tasks.

The study adopted a **review-driven analytical model**, complemented by **two case studies** intended to illustrate the capabilities and limitations of state-of-the-art LLM multi-agent frameworks. Its evaluation criteria were therefore not traditional predictive metrics, but rather **coverage of SDLC tasks, agent collaboration patterns, robustness potential, scalability potential, and identified research gaps**. The paper concluded that role-specialized multi-agent systems offer clear promise for addressing software engineering complexity, especially through decomposition and cross-checking; however, it also found that the field remains fragmented, with unresolved challenges in **trustworthiness, orchestration, benchmarking, and cost control**.

The major strength of this study lies in its **broad software-engineering-specific synthesis**. Its main limitation is that it offers **limited direct empirical validation**.

3.2.2 Study [5]

A highly relevant study for the planning dimension of this thesis is the work of Semanti Basu, Moon Hwan Kim, Semir Tatlidil, Tom Williams, Steven Sloman, and Ruth Iris Bahar, published in Frontiers in Artificial Intelligence (2026) under the title Augmenting Large Language Models with Psychologically Grounded Models of Causal Reasoning for Planning under Uncertainty [2]. The research team, centered at Brown University with collaboration from the Colorado School of Mines, addressed a central weakness of pure LLM reasoning: its difficulty in making reliable planning decisions under uncertainty.

The paper proposed a hybrid reasoning framework in which an LLM planner is combined with a human causal model inside a POMDP-based decision process. The framework was tested on assembly and troubleshooting tasks across seven physical objects using GPT-4o, o3-mini, and o4-mini, and compared against non-LLM planners such as POMCP. The main evaluation criterion was average reward, measured over repeated runs for each object and task. The reported results were highly significant: causal augmentation improved the baseline LLM planners by 305%, 8.2%, and 32% in assembly, and by 57%, 9.6%, and 12% in troubleshooting for GPT-4o, o3-mini, and o4-mini, respectively.

The strength of this study lies in its formal treatment of uncertainty, its explicit use of causal grounding, and its rigorous comparison against both LLM and non-LLM planners. Its main weakness, however, is that the evaluation domain is not software project planning, but rather object-level physical reasoning. Accordingly, while the planning logic is highly relevant to AI PM, the transfer to software planning remains indirect.

3.2.3 Study [6]

In the domain of software effort estimation, Ozan Raşit Yürüm, Hüseyin Ünlü, and Onur Demirörs published An Alternative Software Benchmarking Dataset: Effort Estimation with Machine Learning in the Journal of Systems and Software (2026) [3]. Conducted by a Turkish research team spanning İzmir Bakırçay University and the İzmir Institute of Technology, this study tackled a practical and persistent limitation in effort estimation research: the heavy dependence on proprietary or restricted datasets such as ISBSG.

The authors proposed a free and extendable benchmarking dataset for effort estimation and evaluated it against 732 COSMIC-based ISBSG projects, while their own dataset contained 337 records extracted from 18 studies. They built estimation models using linear regression and nine machine learning algorithms, including Bayesian Ridge, Ridge Regression, Decision Tree, Random Forest, XGBoost, LightGBM, k-Nearest Neighbors, Multi-Layer Perceptron, and Support Vector Regression. The study used leave-one-out cross-validation and an unseen evaluation dataset to assess generalization. The authors reported that machine-learning-based models improved predictive performance over linear regression, and that their smaller open dataset yielded better predictive performance than ISBSG in most cases.

The principal strength of this study is its contribution of a practical open benchmarking resource and its strong comparative methodology across multiple algorithms. Its main limitations are that the dataset remains smaller than ISBSG, is strongly centered on COSMIC size data, and includes fewer contextual project features than a full industrial planning repository might provide.

3.2.4 Study [7]

A study that is particularly relevant to AI PM's input normalization and requirement structuring stage is Structuring Natural Language Requirements with Large Language Models by Johannes J. Norheim and Eric Rebentisch, published in the IEEE Requirements Engineering Workshops (2024) [4]. This work was conducted by the Design Research Group at the University of Strathclyde, with strong systems-engineering orientation.

The study investigated whether GPT-4 can translate free-form natural language requirements into more structured representations using very limited examples. The target representations included EARS-style templates, a custom minimalistic performance modeling language, and a semi-formal LTL-style structure. The core evaluation criterion was the model's ability to consistently transform unstructured requirements into structurally aligned target forms, even under one-shot prompting conditions. The study reported preliminary but promising evidence that GPT-4 could successfully perform this translation when the provided example preserved the same structural pattern as the source requirement.

The strength of this paper lies in its direct relevance to requirements preprocessing, especially for projects like AI PM that begin with raw textual briefs. Its weakness is that it is explicitly a research preview, with limited dataset scale, limited benchmarking depth, and no large industrial validation.

.

3.2.5 Study [8]

In a closely related requirements-engineering study, a system called REQINONE was proposed and published at the 33rd IEEE International Requirements Engineering Conference (2025) [8]. The paper addressed the problem of converting informal project descriptions into structured Software Requirements Specifications (SRS) while reducing hallucination and improving controllability.

REQINONE adopted a modular LLM-agent architecture that decomposed SRS generation into three stages: summary, requirement extraction, and requirement classification. The study evaluated the system using GPT-4o, LLaMA 3, and DeepSeek-R1, and compared the outputs against a holistic GPT-4 baseline as well as entry-level human requirements engineers. Its evaluation criteria included expert judgment of accuracy, structure, and SRS quality, together with classification quality relative to a state-of-the-art classification model. The authors concluded that REQINONE generated more accurate and better-structured SRS documents, and that its classification module achieved comparable or better performance than the state of the art.

The principal strength of REQINONE is its modular decomposition strategy, which strongly resonates with the architectural logic of AI PM. Its limitation is that the study remains focused on SRS generation, not on full downstream project planning artifacts such as dependency graphs, sprint plans, effort reports, or risk

3.2.6 Study [9]

Another highly relevant recent contribution is COLLAB-LLM: A Communication-Centric Role-Based Framework for Scalable Multi-Agent LLM Collaboration, published in the Asian Journal of Research in Computer Science (2026) [9].

This study addressed one of the most pressing issues in multi-agent LLM systems: communication ambiguity and coordination failure. The authors proposed a framework combining a structured communication protocol, a hierarchical role architecture, and a dynamic distributed task-graph engine. The system was evaluated on more than 120 complex multi-step tasks across software engineering, business process automation, and scientific research synthesis. The paper defined success using a task completion threshold above 0.8, and reported overall success rate, communication efficiency, robustness, and scalability. The framework achieved an 89% overall success rate, improving over strong single-agent and multi-agent baselines by 13% to 19%, while scaling to teams of up to eight agents.

The study's key strength lies in its explicit treatment of protocolized agent communication, which is a crucial design idea for any robust agentic planning system. Its main weakness is that it is cross-domain rather than software-planning-specific, and therefore its strong coordination findings must still be adapted carefully to project planning contexts.

3.2.7 Study [10]

Although focused on software security rather than planning, the study LAMPS is highly relevant to the engineering design of multi-agent software systems [10].

LAMPS used four specialized agents coordinated through CrewAI: package retrieval, file extraction, classification, and verdict aggregation. It combined a fine-tuned CodeBERT classifier with LLaMA 3 agents for contextual reasoning. The evaluation used two datasets: a balanced dataset of 6,000 setup.py files and a realistic multi-file dataset of 1,296 files with natural class imbalance. Metrics included accuracy, balanced accuracy, repeated-run evaluation, and McNemar's statistical significance test. The system achieved 97.7% accuracy on the first dataset and 99.5% accuracy / 99.5% balanced accuracy on the second, outperforming MPHunter, RAG-based approaches, and fine-tuned single-agent baselines.

The main strength of this paper is its strong empirical rigor, especially its use of statistical testing and well-defined baselines. Its limitation is that the application domain is security analysis, not project planning. Nevertheless, its agent specialization strategy offers a compelling transferable design pattern for AI PM.

3.2.8 Study [11]

One of the closest recent studies to the conceptual direction of AI PM is QUARE: Multi-Agent Negotiation for Balancing Quality Attributes in Requirements Engineering [11].

QUARE formulated requirements analysis as a structured negotiation process among five quality-specialized agents: Safety, Efficiency, Green, Trustworthiness, and Responsibility, all coordinated by an orchestrator. It further transformed negotiated outputs into KAOS goal models, then applied topology validation and RAG-based verification against standards. The framework was evaluated on five case studies, including MARE, iReDev, and an industrial autonomous-driving specification. The main metrics were compliance coverage, semantic preservation, verifiability, and generated requirement volume. The reported results were highly competitive: 98.2% compliance coverage, 94.9% semantic preservation, 4.96/5 verifiability ،and 25–43% more requirements generated than prior multi-agent RE frameworks.

The study is especially strong because it integrates specialized agents, explicit negotiation, and verification, all of which are conceptually close to AI PM's hybrid philosophy. Its main weakness is that it is currently an emerging 2026 preprint, which means that broader independent replication and long-term external validation are still needed.

3.3 Comparative Synthesis and Research Gap

Taken together, these studies reveal a clear shift in recent software engineering research from monolithic prompting toward role-specialized, modular, and verifiable LLM-based systems. The literature strongly suggests that agent decomposition improves robustness, traceability, and task quality, especially when combined with structured protocols, external knowledge, or formal validation layers. This pattern is evident in modular SRS generation [8], communication-centric coordination [9], role-specialized software analysis [10], and negotiation-based requirements frameworks [11].

At the same time, the reviewed works also expose a significant gap. No single study among the reviewed set offers a complete pipeline that begins with raw software requirements and ends with validated project planning artifacts such as task decomposition, dependency graphs, effort estimation, sprint-oriented planning, and risk-aware summaries within one coherent local-first framework. Some studies are strong in requirements structuring [7][8], some in agent collaboration [4][9][10], some in uncertain planning [5], and others in effort estimation [6], but these capabilities remain largely fragmented.

This gap directly motivates the contribution of AI PM, which aims to unify these strands into an integrated intelligent software project planning system.

3.4 Modern Techniques and Frameworks in the Field

Recent advances in intelligent software engineering have been shaped by the convergence of several major computational paradigms, most notably **Transformer architectures** [1], **Large Language Models (LLMs)** [3], **retrieval-augmented reasoning** [2], **agentic orchestration frameworks** [4], [11], and **graph-based learning methods such as Graph Neural Networks (GNNs)** [12], [13]. Together, these technologies have redefined how software requirements can be interpreted, structured, validated, and transformed into actionable planning artifacts. For a system such as **AI PM**, which aims to convert natural-language project briefs into coherent plans, these paradigms constitute the principal scientific and engineering foundations of the field.

3.4.1 Transformers

The modern wave of AI-driven language systems originates from the Transformer architecture introduced in [1]. Unlike recurrent models, Transformers rely on **self-attention mechanisms** to model contextual relationships between tokens in parallel, enabling them to capture long-range dependencies more efficiently and at scale. This architectural shift is particularly important for requirement documents and planning briefs, which often contain distributed semantic dependencies. In intelligent project planning, Transformers provide the computational basis for identifying functional intent, non-functional constraints, implicit dependencies, and semantic relations across complex requirement narratives.

3.4.2 Large Language Models (LLMs)

Built upon Transformer foundations, **Large Language Models** have become the dominant paradigm for high-level reasoning over natural language. Recent research demonstrates that LLMs can support a wide range of software engineering tasks, including requirements interpretation, code generation, summarization, documentation, and analytical reasoning [3]. Their importance in planning systems stems from their ability to infer latent structure from incomplete or ambiguous input.

However, LLMs are not inherently reliable planning engines. Their outputs may include hallucinated tasks, inconsistent classifications, or missing dependencies [3]. As a result, modern approaches increasingly adopt **hybrid architectures**, where LLM reasoning is supported by validation, retrieval, or structured decomposition rather than being used in isolation.

3.4.3 Retrieval-Augmented Generation (RAG)

One of the most influential frameworks for improving LLM reliability is **Retrieval-Augmented Generation (RAG)**, introduced in [2]. This approach combines language modeling with **external knowledge retrieval**, allowing generated outputs to be grounded in retrieved evidence.

In software planning contexts, RAG enhances consistency by incorporating examples, historical data, and domain-specific patterns. For systems like AI PM, this mechanism reduces unsupported generation and improves the factual and operational relevance of produced plans.

3.4.4 Agentic and Multi-Agent LLM Frameworks

A major trend in recent research is the shift toward **agentic AI and multi-agent systems**. Study [4] highlights that role-specialized agents can distribute complex reasoning tasks across structured workflows, improving modularity and robustness.

Similarly, recent frameworks such as those described in [11] demonstrate how multiple specialized agents can collaborate through structured negotiation and verification mechanisms. This is particularly relevant for planning systems, where tasks such as decomposition, validation, prioritization, and revision must be handled in a coordinated manner.

These approaches show that software planning is not a single-step prediction problem but a **multi-stage reasoning process**, making agent-based architectures highly suitable for systems like AI PM.

3.4.5 Graph Neural Networks and Graph-Based Learning

Another important development is the emergence of **Graph Neural Networks (GNNs)**, which extend deep learning to relational data structures [12]. Unlike sequence-based models, GNNs operate on nodes and edges, making them suitable for modeling dependencies, hierarchies, and interactions.

In software engineering, graph representations are naturally aligned with task dependencies, requirement traceability, and workflow structures. More recent work suggests combining graph reasoning with language models to enhance structural awareness in intelligent systems [13].

In the context of AI PM, graph structures are currently used for **explicit dependency modeling and analysis**, rather than full graph neural training. Nevertheless, GNNs remain a promising future extension for improving prediction accuracy and modeling complex project relationships.

Relevance to the Proposed Project

The AI PM framework integrates these paradigms into a unified architecture. Its reasoning capabilities rely on **Transformer-based LLMs**, its reliability is enhanced through **retrieval mechanisms**, and its design aligns with **multi-agent orchestration principles**. At the same time, its planning logic reflects the importance of **graph-based dependency modeling**.

Overall, the field is evolving toward **hybrid intelligent systems** that combine language understanding, external knowledge, structured validation, and relational reasoning. This evolution provides the theoretical and practical foundation for AI PM as a comprehensive, knowledge-grounded, and dependency-aware planning system.

3.5 Comparative Analysis of Existing Systems

System	Type	Main Features	Core Technologies	Strengths	Weaknesses
Jira + Atlassian Intelligence	Commercial	Timeline planning, Advanced Roadmaps, cross-team dependency tracking, capacity planning, AI-assisted summarization	Cloud project-management platform, internal AI models	Strong enterprise maturity, scalability, and collaboration ecosystem	Does not transform raw natural-language requirements into structured plans; proprietary and cloud-dependent [28]
ClickUp Brain	Commercial	Context-aware AI, summaries, AI agents for workflows	Workspace-aware generative AI, contextual retrieval	Strong integration between docs, tasks, and workflows	Focuses on productivity rather than structured requirement decomposition and planning logic [29]
ReqInOne	Academic	Converts natural language into structured SRS outputs	Modular LLM-based pipeline	Produces structured and accurate requirements	Limited to SRS generation; does not extend to planning artifacts [8]
QUARE	Academic	Multi-agent negotiation, KAOS modeling, verification	Specialized agents, orchestrator, RAG-based validation	Strong verification and requirement-quality reasoning	Does not provide full project planning pipeline [11]

The comparison reveals a clear structural pattern. Commercial systems such as **Jira** and **ClickUp Brain** are effective in collaboration and workflow management but assume that requirements are already structured. Academic systems such as **ReqInOne** and **QUARE** advance requirements engineering but remain limited to the **requirements-analysis layer** rather than full project planning.

3.6 Research Gap and Proposed Innovation

Based on the reference studies and system comparison, several unresolved research gaps remain evident.

First, there is a persistent end-to-end transformation gap.
Recent academic systems significantly improve requirements elicitation, structuring, and verification, yet they do not complete the full transition from raw software briefs to actionable project-management artifacts such as task decompositions, dependency graphs, sprint plans, workload estimates, and risk summaries [8][11]. In other words, current research has made substantial progress in understanding requirements, but not in operationalizing them into executable planning structures.

Second, there is a reliability and validation gap in AI-assisted planning.
The broader literature on LLMs for software engineering shows that generative models remain vulnerable to hallucination, inconsistency, and limited controllability when used in technical workflows [3]. While some recent frameworks mitigate this through modularization or multi-agent negotiation, robust integration between generative reasoning and deterministic acceptance criteria remains underdeveloped. Existing systems often improve output quality, yet they do not fully guarantee structural soundness, traceability, or planning consistency across the entire pipeline.

Third, there is an artifact-integration gap.
Most available systems focus on a single output class: SRS generation, requirement negotiation, workflow management, or task collaboration. They do not unify these functions into a single framework capable of generating multiple interrelated artifacts from the same requirements source. Specifically, the literature still lacks a practical system that jointly supports:

- requirement parsing.
- task decomposition.
- requirement classification.
- dependency modeling.
- effort estimation.
- sprint-oriented organization.
- risk-aware analytical reporting.

3.6.1 Proposed Contribution

Fourth, there is a deployability and transparency gap.
Commercial AI-enabled platforms provide valuable productivity assistance, yet their inner mechanisms are largely proprietary, cloud-dependent, and difficult to audit scientifically [28][29]. This creates limitations for academic reproducibility, privacy-sensitive deployment, and controlled evaluation. For many research and educational environments, there remains a need for a local-first, experimentally inspectable, and methodologically transparent planning system.

In response to these gaps, the proposed innovation of this research is the development of AI PM, a hybrid AI-driven software project planning framework that extends beyond isolated requirements analysis or generic task management. The proposed system introduces an integrated pipeline in which natural-language software requirements are transformed into validated planning artifacts through the combination of:

- LLM-guided reasoning for requirement interpretation and task decomposition
- retrieval-augmented contextual support for grounding planning decisions in reusable knowledge [2],
- deterministic rule-based validation to enforce structure, consistency, and traceability,
- graph-based dependency analysis to model execution order and identify critical paths [12],
- effort estimation and sprint planning to support practical delivery organization [6], risk-aware analytical outputs to improve managerial interpretability.

The novelty of the proposed work therefore lies not in the isolated use of LLMs, but in the hybrid integration of language reasoning, retrieval, validation, and graph-aware planning inside a single coherent framework. This allows AI PM to address a research space that remains insufficiently covered by existing commercial tools and academic studies: the generation of reliable, interpretable, and execution-oriented project plans directly from raw software requirements.

4 Chapter Four: Research Methodology

4.1 Introduction

This chapter presents the research methodology adopted for the design, implementation, and evaluation of the proposed AI PM framework. Since the objective of this study is not limited to theoretical analysis, but extends to the development of a functional intelligent planning system, the methodology combines applied software engineering practice with experimental AI-system research. In this sense, the chapter explains how the study moves from the identified research problem toward a concrete, validated, and operational solution capable of transforming natural-language software requirements into structured project-planning artifacts.

The methodological approach of this research is grounded in the development of a hybrid intelligent system that integrates several complementary techniques, including Large Language Model reasoning, retrieval-augmented support, deterministic rule-based validation, graph-based dependency modeling, and effort-aware planning analysis. Accordingly, the methodology is designed not only to build the system itself, but also to justify the selection of its architectural components, define the workflow through which requirements are processed, and establish the evaluation procedures used to assess the quality and reliability of the produced outputs. This makes the methodology both constructive and analytical, as it addresses system creation and empirical validation simultaneously.

To achieve these goals, the chapter describes the overall research design, the system-development process, the architectural structure of AI PM, the implementation environment, the knowledge-base and planning pipeline construction, and the procedures used for experimentation and evaluation. It also explains how benchmark scenarios, validation criteria, and performance indicators were selected in order to measure planning quality, requirement coverage, dependency coherence, estimation behavior, and overall system robustness. Therefore, this chapter forms the methodological bridge between the theoretical foundations presented earlier and the implementation and results discussed in the subsequent chapters.

4.2 Traditional Approaches to Solving the Software Project Planning Problem

4.2.1 Network-Based Scheduling Methods

One of the earliest and most influential traditional planning approaches was the use of network-based scheduling models, especially the Critical Path Method (CPM) and Program Evaluation and Review Technique (PERT). These methods represent projects as sets of activities connected by precedence relationships, making it possible to compute execution order, identify schedule bottlenecks, and estimate total project duration through path analysis.

The main strength of these methods lies in their mathematical clarity. They provide an explicit mechanism for identifying critical activities, analyzing slack, and organizing schedule control. However, their effectiveness depends heavily on the assumption that tasks and dependencies are already known in advance. In software engineering, this assumption is often unrealistic, because requirements are frequently incomplete and evolving.

4.2.2 Parametric and Size-Based Estimation Models

A second major traditional approach focused on software effort estimation through formal mathematical models such as **COCOMO II** [27], which estimate development effort based on software size and cost drivers.

In parallel, size-based methods such as **Function Point Analysis (FPA)** [26] and **Use Case Points (UCP)** [14] were developed to estimate effort from measurable representations of system functionality. These methods improved planning objectivity by introducing systematic sizing mechanisms.

Despite their importance, these approaches require structured inputs and do not generate full planning artifacts such as task hierarchies, dependencies, or risk-aware plans.

4.2.3 Expert Judgment and Analogy-Based Planning

Another widely used approach is expert judgment, where planning decisions are based on experience or analogy with past projects.

These methods offer contextual awareness but suffer from limitations such as subjectivity, lack of reproducibility, and variability across teams. Research shows that while widely used, their reliability depends heavily on context and calibration practices [15].

4.2.4 Rule-Based and Template-Driven Planning

Traditional systems also relied on predefined templates and rule-based workflows for planning. These approaches improve consistency but lack semantic understanding and cannot interpret unstructured requirements or adapt to new project types.

4.2.5 Agile Heuristics and Iterative Planning

Agile approaches introduced iterative planning mechanisms such as user stories, backlog prioritization, and sprint-based organization. While these improved flexibility, they remain largely human-driven and do not automate the transformation of raw requirements into structured plans.

Before the emergence of modern AI-driven planning systems, the problem of software project planning was typically addressed through a combination of deterministic scheduling methods, algorithmic effort-estimation models, expert-based judgment, and rule-driven project-management workflows. Although these approaches provided important foundations for planning practice, they were generally designed for environments in which requirements were already relatively structured, project scope was stable, and planning decisions could be expressed through fixed formulas or manually engineered heuristics. As a result, they offered partial automation, but they did not fully solve the challenge of transforming raw and ambiguous software requirements into coherent, validated, and adaptive project plans.

4.2.6 Network-Based Scheduling Methods

One of the earliest and most influential traditional planning approaches was the use of network-based scheduling models, especially the Critical Path Method (CPM) and Program Evaluation and Review Technique (PERT). These methods represent projects as sets of activities connected by precedence relationships, making it possible to compute execution order, identify schedule bottlenecks, and estimate total project duration through path analysis. In this paradigm, a project plan is treated as a directed activity network in which planning quality depends on the correctness of task decomposition and dependency specification.

The main strength of these methods lies in their mathematical clarity. They provide an explicit mechanism for identifying critical activities, analyzing slack, and organizing schedule control. For this reason, they became central to classical project planning and remain highly influential in professional project-management standards [28]. However, their effectiveness depends heavily on the assumption that tasks and dependencies are already known in advance. In software engineering, this assumption is often unrealistic, because requirements are frequently incomplete, ambiguous, and evolving. Consequently, CPM and PERT are strong at sequencing predefined work, but weak at deriving that work from unstructured requirement input.

4.2.7 Parametric and Size-Based Estimation Models

A second major traditional approach focused on software effort and cost estimation through formal mathematical models. Among the most widely recognized is COCOMO and its later extension COCOMO II, which estimate development effort and schedule as functions of software size and a set of project-specific cost drivers [27]. Such models represent one of the earliest serious attempts to automate planning decisions through calibrated empirical formulas rather than intuition alone.

In parallel, size-based methods such as Function Point Analysis (FPA) and Use Case Points (UCP) were developed to estimate software effort from measurable representations of system functionality [26][14]. Function Point Analysis measures the functional size delivered to the user, while Use Case Points derive effort indicators from use-case structures, actor complexity, and technical/environmental factors. These methods improved planning objectivity by introducing more systematic sizing mechanisms and reducing total dependence on purely subjective estimation.

Despite their historical importance, these models have notable limitations. First, they require structured input artifacts, such as size measures, stable use cases, or calibrated cost-driver values, which are often unavailable at the early stage of software ideation. Second, they tend to perform best when the project environment resembles the calibration assumptions of the underlying model. Third, while they estimate effort, they do not by themselves produce rich planning artifacts such as validated task hierarchies, dependency graphs, implementation priorities, or risk-aware sprint structures. In other words, they solve the effort-estimation subproblem, but not the broader requirements-to-plan transformation problem.

4.2.8 Expert Judgment and Analogy-Based Planning

Another dominant traditional approach has been expert judgment, in which experienced project managers or development teams estimate effort, schedule, and scope based on prior experience. Related to this is analogy-based estimation, where a current project is planned by comparing it with previously completed projects that appear similar in complexity, domain, or architecture. These methods have remained widely used in industry because they are flexible, intuitive, and often effective when historical knowledge is strong and expert availability is high.

Their key strength is contextual sensitivity. Human experts can often detect subtle domain signals, organizational constraints, and practical implementation challenges that are difficult to encode in rigid mathematical formulas. However, these approaches also suffer from major weaknesses: they are difficult to reproduce, vulnerable to bias, strongly dependent on the availability of experienced estimators, and often inconsistent across teams or organizations. Survey-based literature on software effort estimation repeatedly shows that expert estimation remains common in practice, but its reliability varies significantly with context and calibration discipline [15]. From an automation perspective, expert judgment offers practical wisdom, yet it does not provide a scalable or fully systematic mechanism for intelligent planning.

4.2.9 Rule-Based and Template-Driven Planning

Traditional software planning tools also relied on rule-based workflows and template-driven project decomposition. In such systems, project managers define standard task templates, work breakdown structures, milestone plans, and role allocations based on prior organizational practices. Some degree of automation is achieved by reusing predefined phases such as requirements analysis, design, implementation, testing, and deployment, then assigning durations or responsibilities according to fixed rules.

This approach is useful in repetitive or highly standardized environments, especially where project types share similar life-cycle patterns. It improves consistency and reduces the burden of planning from scratch. Nevertheless, it remains fundamentally limited by its lack of semantic understanding. Rule-based planning can only operate effectively when the problem is already classified into known categories; it cannot deeply interpret ambiguous natural-language requirements, infer hidden dependencies, or dynamically adapt planning logic to novel project structures. Therefore, its automation is procedural rather than intelligent.

4.2.10 Agile Heuristics and Iterative Planning

With the rise of Agile development, planning became increasingly based on iterative heuristics, including user stories, backlog grooming, sprint planning, and relative estimation techniques such as story points. This represented a shift away from heavy up-front deterministic planning toward adaptive short-cycle planning. Agile methods improved responsiveness to change and reduced the rigidity of long-term schedule assumptions.

However, Agile planning remained largely team-driven rather than computationally intelligent. Story-point estimation, for instance, is often subjective and difficult to compare across teams. Moreover, Agile tools generally help organize and track work after requirements have been expressed as backlog items, but they do not automatically transform raw project briefs into validated planning structures [28]. Thus, even though Agile improved flexibility, it did not fully address the automation problem at the semantic front end of software planning.

4.2.11 Limitations of Traditional Approaches

Taken together, the traditional approaches made substantial contributions to software project planning, but they left several key issues unresolved. Most importantly, they assume that planning inputs are already structured, measurable, and sufficiently stable. They also tend to treat scheduling, effort estimation, task decomposition, and risk analysis as partially separate activities rather than as outputs of one integrated reasoning pipeline. Furthermore, they have limited ability to process ambiguous natural-language requirements, weak adaptability to novel project contexts, and insufficient support for knowledge-grounded, traceable, and automatically validated planning outputs.

For these reasons, traditional planning approaches are best understood as foundational but incomplete solutions. They established essential concepts such as task dependencies, critical paths, effort modeling, and decomposition discipline; however, they do not provide the level of semantic interpretation, contextual grounding, and integrated artifact generation required by modern intelligent planning systems. This limitation directly motivates the development of hybrid AI-based frameworks such as AI PM.

4.3 Quantitative Methods (COCOMO II, Function Points, Use Case Points, and PERT)

position because they attempt to transform project characteristics into measurable planning outputs such as effort, duration, and schedule expectations. These methods are based on numerical models, empirical calibration, or formal sizing procedures rather than purely qualitative judgment. Their main objective is to reduce planning subjectivity by introducing systematic estimation mechanisms that can support budgeting, scheduling, and resource allocation. In software engineering, the most widely recognized quantitative approaches include COCOMO II, Function Point Analysis, Use Case Points, and PERT-based estimation [27][26][14][15].

COCOMO II is one of the most established parametric models for estimating software effort, cost, and schedule. It was developed as an evolution of the original COCOMO model to better fit contemporary development practices and modern software life cycles [27]. The model estimates effort primarily as a function of software size and a calibrated set of cost drivers related to product attributes, platform constraints, personnel capability, and project conditions. In simplified form, its effort model may be expressed as:

$$\boldsymbol{PM = A \times Size^{E} \times \prod EMi}$$

where PM denotes person-months, $Size$ represents software size, E is the scale exponent, and $E\mathrm{M}_{\mathrm{i}}$ are effort multipliers. The main strength of COCOMO II lies in its methodological rigor and empirical heritage. However, it assumes that size and cost-driver values can be reasonably identified in advance, which is often difficult during the early requirements stage when project descriptions are still informal or incomplete.

Function Point Analysis (FPA) is another important quantitative method, but unlike COCOMO II, it focuses on measuring the functional size of a software system from the user's perspective [26]. It evaluates software in terms of delivered functionality through categories such as external inputs, external outputs, external inquiries, internal logical files, and external interface files. The underlying principle is that system size can be measured independently of implementation language or development technology. This makes FPA particularly useful for productivity analysis, benchmarking, and cost estimation. Its major strength is the relative objectivity of its sizing philosophy. Nevertheless, FPA requires a sufficiently structured understanding of system functions, which may not be available in early-stage project briefs expressed in free natural language. As a result, while FPA is valuable for standardized environments, it is less effective as a front-end planning mechanism for ambiguous or evolving requirements.

A related method is Use Case Points (UCP), which estimates effort based on the structure and complexity of use cases and actors within a system specification [14]. The classical UCP formulation can be summarized as:

$$\boldsymbol{UCP = (UAW + UUCW) \times TCF \times ECF}$$

where $U\ AW$ is the Unadjusted Actor Weight, $UUCW$ is the Unadjusted Use Case Weight, TCF is the Technical Complexity Factor, and ECF is the Environmental Complexity Factor. The resulting value is then converted into effort through a productivity rate. UCP is particularly attractive because it aligns estimation with use-case-driven analysis and is often easier to apply than full function-point counting in object-oriented or requirements-centered settings. However, the literature also indicates that UCP performance is sensitive to dataset quality, calibration choices, and the subjectivity involved in assigning complexity weights [14]. In addition, like other traditional methods, it depends on having reasonably formalized use-case artifacts, which limits its direct applicability to raw project narratives.

PERT (Program Evaluation and Review Technique) addresses a somewhat different aspect of planning by focusing on time uncertainty rather than software size alone [15]. Instead of producing a single-point estimate for each activity, PERT uses three estimates: optimistic (O) most likely (M), and pessimistic (P) The expected duration is then computed using the weighted average::

$$\boldsymbol{TE = \frac{O + 4M + P}{6}}$$

and the activity variance is often estimated as:

$$\boldsymbol{\sigma 2 = \left(\frac{P - O}{6}\right)^2}$$

This enables planners to reason probabilistically about completion time and schedule risk. The principal strength of PERT is that it explicitly models uncertainty and therefore provides more informative schedule estimates than fixed-duration planning. However, its usefulness still depends on the availability of predefined activities and credible duration judgments. In software projects, where tasks themselves may be uncertain or poorly defined at the outset, PERT can support schedule refinement, but it cannot by itself derive the planning structure from unstructured requirements.

Taken together, these quantitative methods represent major historical advances in software project planning. They introduced formalism, comparability, and measurable reasoning into activities that were previously dominated by intuition alone. Nevertheless, they share a common limitation: they generally assume that planning inputs are already structured, stable, and quantifiable. They do not natively interpret ambiguous natural-language requirements, infer hidden dependencies, or automatically generate integrated planning artifacts. For this reason, although they remain highly valuable as foundational estimation and scheduling techniques, they are insufficient on their own for the broader planning problem addressed in this thesis. This limitation provides a direct rationale for the proposed AI PM framework, which seeks to combine the numerical discipline of traditional methods with the semantic reasoning and adaptability of modern AI-based approaches.

4.4 Machine Learning and Artificial Intelligence Approaches

(Historical Regression, Requirements Classification, and LLM-Based Multi-Agent Systems)

With the growing availability of software project data and the increasing complexity of requirements analysis, software project planning has progressively evolved beyond purely numerical estimation models toward data-driven and AI-assisted methods. Unlike traditional approaches that depend on fixed formulas or manually calibrated parameters, machine learning and artificial intelligence techniques seek to infer planning knowledge from data, language, and interaction patterns. In this context, three important directions have emerged: historical regression for effort estimation, requirements classification, and LLM-based multi-agent systems for higher-level planning and reasoning.

4.4.1 Historical Regression and Data-Driven Effort Estimation

One of the earliest forms of intelligent planning support is the use of historical regression models, in which past software project records are used to learn the relationship between project features and actual effort outcomes. In this paradigm, effort estimation is treated as a supervised prediction problem:

$$\hat{E} = f(x_1, x_2, \dots, x_n)$$

where $\hat{E}$ denotes the predicted effort and $(x_1, x_2, \dots, x_n)$ represent project attributes such as size, complexity, functional scope, team factors, or other measurable indicators. Early implementations relied mainly on linear regression, whereas later work incorporated more expressive machine-learning models such as Decision Trees, Random Forests, Support Vector Regression, XGBoost, LightGBM, and Multi-Layer Perceptrons.

A recent and highly relevant study demonstrated that machine-learning models can outperform classical regression under appropriate data conditions [6]. Using both ISBSG and an alternative open benchmarking dataset, the authors compared linear regression with nine machine-learning algorithms and showed that the predictive performance of data-driven models varies according to dataset characteristics and evaluation method, but can exceed traditional baselines in many cases. This result is important because it confirms that software effort estimation is not purely a formula-driven exercise; rather, it can benefit substantially from learned patterns extracted from historical project data.

The strength of historical regression methods lies in their empirical grounding and their ability to capture nonlinear relations that fixed estimation formulas may miss. However, their limitations are equally significant. Their performance depends heavily on the quality, size, and representativeness of the training data. They may also struggle with early-stage planning when project descriptions are still incomplete or primarily textual rather than numerical. Consequently, while historical regression improves estimation accuracy, it does not by itself solve the broader problem of interpreting raw requirements and producing integrated planning artifacts.

4.4.2 Requirements Classification

A second major direction in AI-assisted planning is requirements classification, which focuses on automatically organizing textual requirements into meaningful categories such as functional requirements, non-functional requirements, quality attributes, priority levels, or structural requirement classes. This task is especially important because software planning quality depends heavily on whether the system can correctly distinguish between different requirement types before decomposition, scheduling, and estimation take place.

Earlier approaches to requirements classification relied on traditional natural language processing and classical machine learning, often using handcrafted features, lexical patterns, and supervised classifiers. More recently, Large Language Models have significantly changed this landscape by enabling zero-shot, few-shot, and modular reasoning over complex requirements text. A strong recent example is ReqInOne [8]. ReqInOne decomposes software requirements specification generation into three stages: summary generation, requirement extraction, and requirement classification, each supported by tailored prompts. According to the reported results, the system produced more accurate and better-structured SRS documents than a holistic GPT-4-based baseline, while its classification module achieved performance comparable to or better than the state of the art.

The importance of this direction is also supported by broader literature on LLMs in software engineering, which shows their growing use across requirements engineering tasks, including classification, elicitation, and validation [3]. This indicates that the field has moved from purely syntactic text processing toward more semantically informed language reasoning. The primary strength of AI-based requirement classification lies in its ability to reduce ambiguity, improve traceability, and structure unorganized text before downstream planning. Nevertheless, these systems remain vulnerable to label instability, prompt sensitivity, hallucinated interpretations, and limited robustness when domain context is weak or noisy.

4.4.3 LLM-Based Multi-Agent Systems

The most advanced recent direction in intelligent software planning is the emergence of LLM-based multi-agent systems, in which multiple language-model agents collaborate through specialized roles rather than relying on a single monolithic prompt. In such systems, one agent may act as a planner, another as a critic, another as a validator, and another as a monitor or risk analyzer. This architectural shift reflects the recognition that complex software-engineering tasks are rarely solved reliably by one generative step; instead, they require decomposition, review, correction, and structured coordination.

A major synthesis of this direction was presented in a comprehensive study reviewing multi-agent LLM systems across the software development life cycle [4]. This work concluded that role-specialized collaboration can improve robustness, scalability, and autonomous problem-solving in software engineering. Its significance lies in framing multi-agent orchestration as a methodological transition rather than an isolated experimental technique.

A more concrete example is QUARE [11]. QUARE formulates requirements analysis as a structured negotiation process among multiple specialized agents, coordinated by an orchestrator and supported by topology validation and retrieval-augmented verification. The reported results show strong compliance coverage, semantic preservation, and verifiability, illustrating that multi-agent decomposition can improve both analytical depth and structural reliability. This is especially important for planning systems because requirements often contain internal conflicts, implicit trade-offs, and mixed quality concerns that are difficult for a single model invocation to resolve consistently.

The key strength of LLM-based multi-agent systems lies in their modularity. By distributing reasoning across explicit roles, they make the planning process more interpretable, more controllable, and often more reliable than monolithic prompting. However, they also introduce new challenges, including orchestration complexity, communication overhead, cost accumulation, and the need for robust coordination protocols. Therefore, while they represent one of the most promising directions in intelligent software planning, they are most effective when integrated with additional mechanisms such as retrieval, validation, and deterministic safeguards.

4.4.4 Relevance to the Proposed Project

The proposed AI PM framework is conceptually aligned with all three of these directions, but extends them into a more integrated planning pipeline. From historical regression and machine learning, it inherits the importance of data-grounded effort estimation. From requirements classification, it adopts the need to structure and normalize textual input before planning. From LLM-based multi-agent systems, it adopts the principle of role separation, critique, and staged reasoning. However, AI PM does not stop at any one of these layers in isolation. Instead, it combines them with retrieval-augmented support, graph-based dependency analysis, and deterministic validation in order to address a broader problem: the generation of reliable, execution-oriented project plans from raw natural-language software requirements.

4.5 Identification of the Research Gap and Unmet Requirements:

The review of traditional planning methods and recent AI-based approaches reveals that substantial progress has been made in software effort estimation, requirements analysis, and LLM-assisted reasoning; however, these advances remain fragmented across separate problem layers. Classical quantitative methods such as COCOMO II, Function Point Analysis, Use Case Points, and PERT provide useful estimation and scheduling mechanisms, yet they assume that project inputs are already structured and measurable [27][26][14][15]. They do not interpret ambiguous natural-language requirements, infer latent task structures, or generate integrated planning artifacts from raw project descriptions. Consequently, they address only isolated planning subproblems rather than the full requirements-to-plan transformation challenge.

More recent machine learning and AI approaches partially reduce these limitations, but they still leave important gaps unresolved. Historical regression and machine-learning estimation models improve predictive effort accuracy when sufficient project data are available, as demonstrated in recent work on ML-based effort estimation [6]. Nevertheless ،these models remain strongly dependent on curated historical datasets and numerical project descriptors; they do not, on their own, interpret requirements text or produce dependency-aware project structures. Similarly, modern requirements-classification systems such as ReqInOne improve the structuring of software requirements specifications through modular LLM reasoning [8], but they stop at the SRS generation and classification level. They do not extend their outputs into effort estimation, execution sequencing, sprint planning, or project risk analysis.

At a more advanced level, LLM-based multi-agent systems have introduced modular and role-specialized reasoning into software engineering workflows. A comprehensive literature review shows that multi-agent LLM systems improve robustness and decomposition for complex software-engineering tasks [4], while systems such as QUARE demonstrate strong results in quality-aware requirements negotiation and verification [11]. Yet even these systems remain focused on specific reasoning tasks such as negotiation, specification quality, or requirements balance. They do not provide a complete, operational pipeline that transforms raw project briefs into validated project-management artifacts suitable for execution monitoring and practical planning.

Based on this analysis, the central research gap can be stated as follows:

Current studies do not provide a unified, trustworthy, and practically deployable framework that converts natural-language software requirements into validated, dependency-aware, estimation-ready, and execution-oriented project plans within a single coherent pipeline.

This research gap can be decomposed into several more specific unmet requirements, each of which is highly relevant to the proposed project.

4.5.1 Unmet Requirement 1: End-to-End Transformation from Requirements to Plan

Most existing studies address only one stage of the planning lifecycle. Some focus on effort estimation, others on requirements classification, and others on agent collaboration or quality negotiation. What remains insufficiently solved is the end-to-end transformation of raw software requirements into a complete planning structure that includes tasks, dependencies, effort estimates, sprint grouping, and analytical summaries. In other words, there is still no widely established framework that operationalizes requirements directly into executable project planning artifacts.

4.5.2 Unmet Requirement 2: Reliable Integration of LLM Reasoning with Deterministic Validation

The literature consistently reports that LLMs are powerful in reasoning over natural language but vulnerable to hallucination, inconsistency, and prompt sensitivity [3][5]. Existing systems improve specific outputs through modularization or multi-agent collaboration, but few offer a robust mechanism that combines generative planning with deterministic structural validation, schema enforcement, traceability checks, and graph-consistency verification. This leaves a critical trustworthiness gap, especially in technical planning contexts where plausible but invalid outputs can cause serious downstream errors.

4.5.3 Unmet Requirement 3: Dependency-Aware Planning Beyond Flat Task Generation

Many AI-assisted systems generate textual summaries, requirement lists, or isolated tasks, but they do not model the dependency structure of project execution in a rigorous manner. Without explicit dependency analysis, it becomes difficult to identify critical paths, bottlenecks, sequencing constraints, and parallelizable work packages. Therefore, there is an unmet need for planning systems that move beyond flat decomposition toward graph-aware execution modeling [12].

4.5.4 Unmet Requirement 4: Joint Support for Multiple Interrelated Planning Artifacts

Existing approaches tend to optimize a single artifact class, such as an SRS document, an estimate, or a conversational answer. In practice, however, project planning requires several connected artifacts, including:

- normalized requirements,
- structured task lists,
- requirement classifications,

- effort estimates,
- dependency graphs,
- sprint plans,
- risk summaries,
- reviewable planning reports.

The absence of such integration limits both practical usefulness and academic completeness. A major unmet requirement is therefore the generation of multiple coherent planning outputs from the same input source.

4.5.5 Unmet Requirement 5: Deploy ability in Resource-Constrained and Privacy-Sensitive Settings

Many modern commercial AI platforms depend on proprietary cloud ecosystems, opaque internal models, or high-cost infrastructure [28][29]. Likewise, some research systems are evaluated in controlled environments without strong consideration for local deployment, reproducibility, or operational transparency. This creates a gap for local-first or resource-aware intelligent planning systems that can be deployed in educational, research, or small-team settings without sacrificing methodological clarity.

4.5.6 Unmet Requirement 6: Transparent and Measurable Evaluation of Planning Quality

Although AI-assisted planning research is growing rapidly, evaluation remains inconsistent across studies. Some works assess linguistic quality, others measure classification accuracy, and others focus on case-specific outcomes. There is still a need for a framework in which planning quality is measured through a structured set of criteria such as requirement coverage, classification consistency, dependency coherence, estimation behavior, and structural validity. Without such evaluation, it is difficult to compare systems rigorously or determine whether a generated plan is genuinely usable.

4.5.7 Implication for the Proposed Research

These unmet requirements collectively define the precise motivation for the proposed AI PM framework. The project is designed to address this gap by introducing a hybrid intelligent planning pipeline that combines:

- LLM-based requirement interpretation,
- retrieval-augmented contextual grounding [2],
- deterministic validation rules,
- dependency-graph construction [12],
- effort-aware planning logic [6],
- sprint-oriented organization,
- and risk-sensitive analytical reporting.

Accordingly, the contribution of this research is not merely the use of AI in software planning, but the construction of a trustworthy, integrated, and execution-oriented planning framework capable of bridging the gap between raw software requirements and practical project-management artifacts. This directly addresses the limitations left unresolved by both traditional estimation models and recent AI-based studies.

4.6 Data Flow and Agent Interaction

The effectiveness of the proposed architecture depends not only on the presence of multiple computational components, but also on the manner in which these components exchange information, refine intermediate outputs, and collectively transform raw requirements into validated planning artifacts. For this reason, the proposed AI PM framework is best understood as a staged data-flow pipeline in which each component receives a structured input, performs a specialized transformation or analysis, and produces an output that becomes part of the input context for the subsequent stage. This design promotes modularity, traceability, and controlled reasoning, while reducing the likelihood of error propagation across the planning workflow [4][16][17].

At a high level, the system operates through the following sequence:

1. **Requirements ingestion**
2. **Requirement normalization and parsing**
3. **Planning generation through a hybrid planner**
4. **Knowledge-grounded effort enrichment**
5. **Critic-based structural review**
6. **Dependency-graph construction and analytics**
7. **Sprint planning and delivery grouping**
8. **Risk analysis**
9. **Monitoring and progress interpretation**
10. **Persistence of output artifacts and evaluation traces**

This staged pipeline may be expressed abstractly as:

$$D_{out} = M_n\left(M_{n-1}\left(\dots M_2\left(M_1(D_{in})\right)\dots\right)\right)$$

where (D_{in}) is the raw project input, (M_i) represents a transformation module, and (D_{out}) is the final set of planning artifacts. Such a formulation emphasizes that the system is not a single inference step, but rather a chain of constrained computational transformations. The complete directional flow between modules, including the auxiliary support provided by the knowledge base and the convergence of late-stage analytics, is illustrated in Figure 3.1.

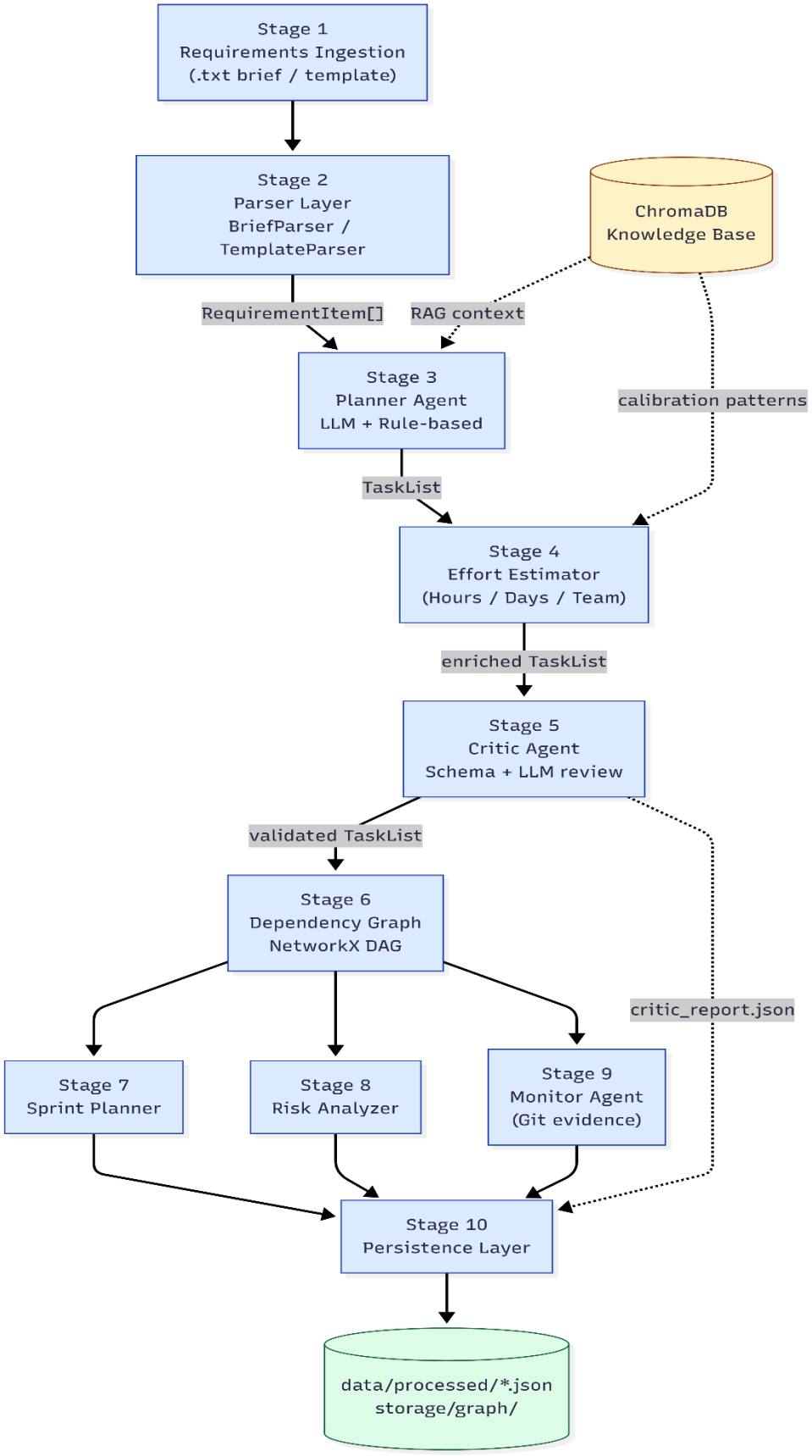

Figure 3.1: Staged data-flow architecture of the AI PM pipeline. Solid arrows indicate primary artifact flow between sequential stages; dashed arrows indicate auxiliary outputs that

are persisted independently or contextual support drawn from the knowledge base. The Planner (Stage 3) and the Effort Estimator (Stage 4) both interact with the ChromaDB knowledge base as a cross-cutting reasoning resource. Late-stage analytics (Stages 7–9) operate in parallel on the validated dependency graph and converge into the unified persistence layer (Stage 10).

4.6.1 Stage 1: Input Acquisition and Parsing

The first stage begins when the user submits either a narrative project brief or a structured requirement template. This input is processed by a parser specialized for the input format, producing normalized requirement units associated with traceable identifiers. The goal of this stage is to eliminate direct dependence on raw document form and establish a clean internal representation that can be reasoned over computationally [7][8].

The output of this stage is a structured sequence of requirement items:

$$R = \{r_1, r_2, \dots, r_n\}$$

where each requirement is associated with a source reference. These source identifiers later support traceability between generated tasks and original requirements.

4.6.2 Stage 2: Planner Agent Interaction

The parsed requirements are then passed to the Planner Agent, which functions as the primary decomposition and reasoning component. Internally, the planner may follow one of two execution paths:

an LLM-guided path, in which the normalized requirements are combined with retrieved knowledge-base context and passed to the local language model;

a rule-based fallback path, activated when the LLM is unavailable, fails, or produces output that cannot be accepted safely.

This interaction is important because it makes the planning stage adaptive without sacrificing reliability. The planner is therefore not a static generator, but a controlled reasoning component whose output is mediated by external context and architectural safeguards [2][16].

The planner produces a preliminary TaskList, in which each task contains fields such as:

- identifier,
- title,
- description,
- requirement type,
- complexity level,
- dependencies,
- source trace,
- optionality,
- and confidence level.

4.6.3 Stage 3: Knowledge-Base Interaction

When knowledge-base support is enabled, the planner queries a ChromaDB vector store containing planning examples, estimation patterns, and calibration records [20]. The retrieved context is injected into the planning stage to improve the semantic grounding of generated tasks. This means that the planner does not reason in isolation, but in interaction with a contextual memory layer, consistent with retrieval-augmented reasoning paradigms [2].

The same knowledge base also supports the Effort Estimator, which uses retrieved records to softly calibrate rule-based effort estimates. In this way, the knowledge base participates in two distinct but related interactions:

- planning augmentation, by improving task decomposition quality;
- estimation calibration, by adjusting effort values using relevant prior examples.

4.6.4 Stage 4: Critic Agent Review

Once the planner produces a candidate task list, the result is forwarded to the Effort Estimator, which assigns quantitative planning metadata to each task before any structural review takes place. This stage transforms the plan from a structural decomposition into a workload-aware execution artifact, consistent with data-driven and benchmark-grounded estimation approaches [6][14][15][18].

For each task, the estimator produces an estimated number of hours, an equivalent number of days, a recommended team size, and an associated skill profile. Estimation is performed through a calibrated blend of rule-based heuristics and retrieval-augmented evidence drawn from the knowledge base, where the relative weight of each source varies as a function of task complexity. This hybrid approach grounds estimates in historical evidence without requiring a custom-trained regression model.

At this point, the interaction pattern between components becomes cumulative:

- the planner provides task semantics,
- the knowledge base contributes calibration evidence,
- the estimator adds execution-level numerical attributes that downstream stages will subsequently consume.

4.6.5 Stage 5: Effort and Resource Enrichment

After effort enrichment is complete, the quantitatively annotated task list is submitted to the Critic Agent. This interaction introduces a second layer of intelligence into the pipeline. Rather than assuming that the planner's output — even when enriched with effort metadata — is final, the system subjects it to structured review [4][16][11].

The critic evaluates the plan through a logic-driven validation stage and, optionally, an additional LLM-based reasoning stage. Its role is to detect:

- malformed task structure,
- invalid or missing dependencies,
- cyclic references,
- implausible FR/NFR balance,
- bundled NFR concerns,
- weak effort completeness,
- and other structural inconsistencies.

The output of this interaction is a CriticReport containing a score, an issue list, and a decision label. Plans that fail review are either rejected outright or returned for revision before they can propagate to subsequent stages. This ordering — generate → enrich → validate — ensures that the critic reasons over the complete task representation, including its quantitative attributes, rather than over an incomplete intermediate artifact. The review stage therefore preserves planning transparency and makes quality control an explicit architectural checkpoint rather than an implicit assumption.

4.6.6 Stage 6: Dependency Graph Construction

The validated and enriched task list is then passed to the **Dependency Graph Engine**, which converts task dependencies into a directed graph representation [12][21]. This interaction is critical because it transforms planning from a flat textual artifact into a formal relational model.

The graph engine derives:

- root tasks,
- leaf tasks,
- bottleneck tasks,
- parallel execution groups,
- critical-path sequences,
- and structural validation warnings.

4.6.7 Stage 7: Sprint Planner Interaction

The graph analytics are then consumed by the **Sprint Planner**, which organizes tasks into delivery slices based on dependency generations, effort load, and thematic coherence [28]. This stage illustrates a higher-order interaction pattern in the architecture: later modules do not return to the raw input, but operate on the progressively enriched outputs of previous components.

4.6.8 Stage 8: Risk Analyzer Interaction

The **Risk Analyzer** then inspects both the structured task set and the analytical summary generated by earlier stages. It evaluates the plan in terms of:

- bottleneck concentration,
- critical-path complexity,
- overall effort pressure,
- dependency-chain fragility,

- role concentration,
- and critic-detected quality concerns.

This stage aligns with recent research emphasizing uncertainty-aware reasoning in planning systems [5].

4.6.9 Stage 9: Monitor Agent Interaction

A further architectural layer is provided by the **Monitor Agent**, which links the generated planning artifacts to implementation evidence such as Git commit history. This creates an important bridge between **planning-time intelligence** and **execution-time observation**.

This interaction extends the architecture beyond one-time plan generation, supporting supervision, progress interpretation, and schedule awareness.

4.6.10 Agent Coordination Logic

Although the proposed system is not a conversational multi-agent simulation in the narrowest sense, it does embody a **role-specialized agentic workflow** [4][16][17]. Each agent-like component has a defined responsibility:

- the **Planner Agent** generates and decomposes,
- the **Critic Agent** reviews and validates,
- the **Risk Analyzer** interprets vulnerabilities,
- the **Monitor Agent** interprets implementation evidence.

This separation of concerns offers several advantages:

- improved modularity,
- clearer debugging and evaluation,
- better traceability of decisions,
- reduced dependence on one monolithic model invocation,
- and easier insertion of fallback or validation logic.

4.6.11 Persisted Artifacts and Traceability

A critical design choice in the proposed data flow is that each major stage produces explicit persisted outputs. These include:

- `tasks.json`
- `plan_summary.json`
- `dependency_graph.json`
- `critic_report.json`
- `risk_report.json`
- `monitor_report.json`

This persistence model has strong methodological value. It makes the pipeline auditable, facilitates empirical evaluation, supports reproducibility, and allows downstream interfaces to consume planning outputs independently of the generation process. In research terms, it also enables the system to function as an **observable experimental pipeline** rather than a black-box intelligent assistant.

4.6.12 Methodological Significance

The proposed data-flow and agent-interaction model is significant because it operationalizes hybrid AI planning as a **structured computational process** rather than a one-step prediction task. By decomposing planning into specialized stages with explicit artifacts, the framework improves reliability, interpretability, and scientific measurability. This aligns with the broader transition toward modular and agent-based AI systems in software engineering research [4][16].

4.7 Detailed Architectural Structure and Components

4.7.1 The Four Agents, the Local LLM, and the Vector Knowledge Base

The proposed AI PM architecture is designed as a hybrid local-first intelligent planning system composed of several tightly coordinated components, each of which performs a distinct analytical or reasoning function. Rather than relying on a single monolithic AI module, the architecture distributes responsibility across a set of specialized agents and support modules. This design improves interpretability, modularity, fault tolerance, and experimental traceability. This direction is consistent with recent work on LLM-based multi-agent systems and role-specialized agent collaboration in software engineering [4], [16], [17].

At a conceptual level, the architecture may be represented as:

$$\boldsymbol{A = I, P, L, K, C, E, G, S, R, M, O}$$

where:

$\boldsymbol{I}$ denotes the input and parsing layer,

$\boldsymbol{P}$ denotes the Planner Agent,

$\boldsymbol{L}$ denotes the local LLM layer,

$\boldsymbol{K}$ denotes the vector knowledge base,

$\boldsymbol{C}$ denotes the Critic Agent,

E denotes the Effort Estimator,

G denotes the Dependency Graph Engine,

S denotes the Sprint Planner,

R denotes the Risk Analyzer,

M denotes the Monitor Agent,

O denotes the persisted output artifacts.

Although the entire architecture contributes to system performance, the most critical research components are the four agentic modules, the local LLM reasoning engine, and the vector-based knowledge layer.

4.7.2 Input and Parsing Layer

The architectural entry point is the input and parsing layer, which accepts either a free-form project brief or a structured requirement template. This layer is implemented through specialized parsers that normalize the incoming text into traceable requirement items. The result is a set of requirement units that can be consumed consistently by the planning logic. This stage is foundational because it separates the external document format from the internal reasoning representation, thereby ensuring that downstream modules operate on well-defined requirement objects rather than raw text alone. This aligns with requirements engineering principles that emphasize structured, traceable, and analyzable requirements representations [38], as well as recent work on using LLMs to structure natural-language requirements [7], [8].

4.7.3 Planner Agent

The Planner Agent is the central reasoning component of the architecture and the first of the four principal agentic modules. Its primary function is to transform normalized software requirements into a structured task plan. Each generated task includes a unique identifier, an action-oriented title, a description, a requirement type label, a complexity rating, a dependency list, and a source reference that preserves traceability back to the originating requirement.

From an architectural perspective, the Planner Agent performs three major functions:

semantic interpretation of the requirement text,

task decomposition into manageable planning units,

initial structural organization of the resulting task set.

The planner is designed to work under a hybrid execution mode. When the local LLM is available and operational, it performs LLM-guided planning enriched by retrieved contextual knowledge. When the model is unavailable or its output cannot be accepted safely, the

planner falls back to rule-based planning logic. This hybridization is an important architectural feature because it combines expressive reasoning with operational robustness. The use of LLMs for reasoning over software engineering tasks is supported by recent surveys and studies on LLM-assisted software engineering and requirements generation [3], [4], [7], [8].

4.7.4 Local LLM Layer

The planning intelligence of the system is supported by a local Large Language Model layer implemented through Ollama, using a private planning model based on the Qwen family [19]. The role of this layer is not to serve as an unrestricted conversational model, but rather as a structured planning reasoner that operates within controlled prompts and produces machine-readable JSON outputs. This controlled use of LLM reasoning is grounded in the broader development of transformer-based language models, few-shot learning, and prompting-based reasoning [1], [31], [32].

Several architectural characteristics make this local LLM layer significant:

it operates locally, improving privacy and deployment independence;

it is configured for low-temperature deterministic output, which supports reproducibility;

it adapts context and completion size dynamically based on prompt length;

it attempts structured JSON extraction and sanitization before any output is accepted.

This means that the LLM layer is not treated as a black box. Instead, it is embedded within a controlled execution interface that monitors latency, retries under memory constraints, and cleans model output before it enters the rest of the pipeline. The local-first nature of this component is especially important in educational and research environments where privacy, offline operation, and experimental auditability are desirable.

4.7.5 Vector Knowledge Base

A second core architectural component is the vector knowledge base, implemented using ChromaDB with persistent local storage [20]. This module stores semantically indexed planning knowledge in the form of curated records such as:

planning examples,

historical estimation records,

task-level estimation patterns,

and calibration points derived from classical estimation logic such as COCOMO-style baselines [27], [34].

The knowledge base uses dense semantic embeddings generated by a SentenceTransformer model, specifically all-MiniLM-L6-v2, to support similarity-based retrieval [25]. When

queried with a requirement or task description, it returns the most relevant stored records together with metadata and distance scores. Architecturally, this component performs two essential roles:

it supplies retrieval-augmented context to the Planner Agent,

and it supports effort calibration in the estimation layer.

Thus, the vector knowledge base functions as an external memory system that grounds planning decisions in reusable domain knowledge. This reduces hallucination risk and helps the architecture move beyond purely zero-shot generation. This design is conceptually aligned with retrieval-augmented generation, where external knowledge retrieval is used to improve the grounding and factual reliability of model outputs [2].

4.7.6 Critic Agent

The Critic Agent is the second major agentic component and serves as the primary validation and review layer of the architecture. Its responsibility is to inspect the task plan generated by the Planner Agent and determine whether it is structurally, logically, and analytically acceptable.

The Critic Agent performs validation across two levels:

a logic-based validation layer, which checks for schema correctness, duplicate identifiers, invalid dependencies, cyclic relations, missing estimates, suspicious FR/NFR ratios, and problematic bundling of non-functional concerns;

an optional LLM-assisted review layer, which performs high-level reasoning about missing scope, unrealistic complexity distribution, and potential plan-quality issues.

The output of this component is a structured CriticReport containing:

a status label, approved, needs_revision, or rejected,

a numerical score,

a list of issues,

and improvement suggestions.

Within the architecture, the Critic Agent is crucial because it prevents the system from treating generated plans as inherently valid. It introduces a formal acceptance barrier that improves reliability and makes planning quality measurable. This role is consistent with multi-agent LLM architectures in which different agents perform generation, review, critique, or coordination responsibilities [4], [16], [17].

4.7.7 Effort Estimator and Execution Enrichment

After validation, the plan is passed to the Effort Estimator, which enriches each task with execution-oriented metadata such as estimated hours, estimated days, recommended team size, required skill, suggested owner role, and potential risk flags. This module combines:

complexity-based baseline rules,

requirement-type multipliers,

and RAG-supported calibration from the vector knowledge base.

As a result, the architecture does not merely generate tasks; it transforms them into actionable planning units that can support scheduling, staffing, and sprint design. This estimation logic is related to established software effort estimation research, including machine-learning-based effort estimation, use-case-point estimation, traditional cost modeling, and software estimation practice surveys [6], [14], [15], [18], [34], [36].

4.7.8 Dependency Graph Engine

The Dependency Graph Engine is responsible for transforming the enriched task list into a formal directed graph representation. Each task becomes a node, and each dependency becomes a directed edge. This module computes:

validity of the graph as a DAG,

root and leaf tasks,

bottleneck tasks,

parallel execution groups,

and the critical path.

Architecturally, this component is essential because it converts the plan from a flat list into a relational execution model. The graph representation also provides analytical signals to later modules, especially the Sprint Planner and Risk Analyzer. The implementation of this type of graph analysis is supported by graph-processing tools such as NetworkX [21], while the use of critical-path reasoning is rooted in classical project scheduling methods [33].

4.7.9 Sprint Planner

The Sprint Planner is a supporting component that consumes graph generations and effort values in order to group tasks into sprint-level delivery slices. Each sprint includes a name, goal, task group, duration estimate, total hours, and owner-role summary. This component

makes the architecture more practically relevant by converting the analytical plan into delivery-oriented structure rather than leaving it at the level of abstract task decomposition. This is consistent with agile software planning principles and project management guidance that emphasize iterative delivery, sprint organization, and structured planning outputs [35], [37].

4.7.10 Risk Analyzer

The Risk Analyzer is the third major agentic module. It reasons over the validated and enriched planning outputs in order to identify project vulnerabilities. Its analysis spans multiple dimensions, including:

bottlenecks,

high-complexity critical-path tasks,

excessive effort load,

dependency fragility,

team allocation pressure,

and critic-detected quality concerns.

The output of this component is a RiskReport containing severity levels, mitigation suggestions, and an aggregate project risk score. In architectural terms, it acts as a synthesis layer that interprets the outputs of several upstream modules and translates them into managerial insight. This role is aligned with project management practices that treat risk identification, analysis, and mitigation as essential planning activities [37].

4.7.11 Monitor Agent

The Monitor Agent is the fourth principal agentic component. Unlike the Planner, Critic, and Risk Analyzer, which primarily operate during plan generation and evaluation, the Monitor Agent extends the system into the execution stage. It analyzes implementation evidence, particularly Git commit history, to infer task progress and compare actual work against the planned task structure.

Its output is a MonitorReport that summarizes:

overall progress,

number of completed and in-progress tasks,

commit evidence matched to tasks,

and tasks that may be behind schedule.

Architecturally, this component is important because it connects the planning framework to real development activity, thereby transforming the system from a static planning generator into a planning-and-monitoring environment. This supports the broader project management principle that planning should be connected to monitoring, control, and evidence-based progress assessment [37].

4.7.12 Component Interaction Logic

The architectural flow among these components is sequential but interdependent. The Planner Agent depends on the local LLM and the vector knowledge base for contextual reasoning. The Critic Agent validates the planner's output. The Effort Estimator and Dependency Graph Engine enrich the validated plan structurally and quantitatively. The Sprint Planner organizes delivery, the Risk Analyzer evaluates vulnerabilities, and the Monitor Agent later compares the plan against implementation evidence. This staged interaction ensures that no single component bears the full cognitive burden of planning. Such decomposition reflects the broader direction of multi-agent LLM systems, where complex tasks are distributed among specialized agents that communicate, validate, and refine outputs collaboratively [4], [16], [17].

4.7.13 Architectural Significance

The detailed architecture of AI PM is significant because it reflects a deliberate shift from monolithic AI generation toward hybrid, role-specialized, and auditable intelligent planning systems. The four-agent design improves functional separation, the local LLM ensures private and controllable reasoning, and the vector knowledge base provides domain grounding without requiring costly retraining. Together, these components create an architecture that is both academically rigorous and practically deployable for requirements-driven software project planning. This architectural direction is supported by recent advances in LLM-assisted software engineering, retrieval-augmented generation, structured requirements processing, and multi-agent LLM collaboration [2], [3], [4], [7], [8], [16], [17].

4.8 Evaluation Function, Penalty Scheme, and Adopted Inference Algorithms

The proposed AI PM framework does not rely on a single binary notion of correctness. Instead, it adopts a layered decision model in which planning outputs are assessed through a set of complementary evaluation functions, explicit penalty schemes, and hybrid inference algorithms. This design reflects the nature of the problem itself: software project planning is not merely a generation task, but a constrained reasoning process in which semantic

adequacy, structural validity, estimation quality, and execution feasibility must all be considered simultaneously [37], [38].

Evaluation Function

The principal evaluation function of the system is defined as a multi-metric planning score rather than a single-task accuracy measure. Let the generated plan be denoted by Y. The quality of Y is evaluated through four primary dimensions:

$$\boldsymbol{F(Y)} =) \frac{\boldsymbol{4C_{cov}(Y)} + \mathrm{C_{CLS}}(\boldsymbol{Y}) + \boldsymbol{C_{cmp}(Y)} + \boldsymbol{C_{dep}(Y)}}{4}$$

where:

$\boldsymbol{C_{cov}}$ is the requirement coverage score,

$\mathrm{C_{CLS}}$ is the classification consistency score,

$\boldsymbol{C_{cmp}}$ is the complexity calibration score,

$\boldsymbol{C_{dep}}$ is the dependency coherence score.

This multi-dimensional evaluation design is suitable for requirements-driven planning because it evaluates completeness, classification quality, estimation structure, and dependency validity rather than treating planning as a simple text-generation problem [7], [8], [38].

4.8.1 Requirement Coverage

Requirement coverage measures how many source requirements are represented in the generated plan:

$$\mathrm{C_{COV}}(Y) = \frac{|R_{covered}|}{|R|}$$

where R is the set of normalized requirements and $R_{covered}$ is the subset for which at least one valid generated task exists. This metric captures semantic completeness and penalizes omission. The use of coverage is aligned with requirements engineering principles, where traceability between source requirements and derived artifacts is essential for validation and auditability [38].

4.8.2 Classification Consistency

Because the system distinguishes between functional requirements (FR) and non-functional requirements (NFR), it evaluates whether the FR/NFR ratio of the generated plan is close to the expected distribution:

$$C_{CLS}(Y) = \max\left(0,\ 1 - \frac{\left|FR_{actual} - FR_{expected}\right|}{0.3}\right)$$

This score allows moderate deviation, but penalizes plans whose classification balance diverges strongly from the intended requirement profile. This is relevant because structured requirements analysis commonly requires distinguishing functional behavior from quality attributes and constraints [38].

4.8.3 Complexity Calibration

In the current implementation, the final complexity score is based on the normalized Shannon entropy of the generated complexity distribution:

$$C_{cmp}(Y) = \frac{-\sum_{i=1}^{k} p_i \log_2 p_i}{\log_2 5}$$

where$\boldsymbol{pi}$ is the proportion of tasks assigned to complexity level i, and the denominator normalizes the score by the maximum entropy over five complexity classes. This formulation penalizes unrealistic uniformity and rewards meaningful variation in task difficulty. The motivation for evaluating task complexity is consistent with software effort estimation literature, where effort prediction depends strongly on size, complexity, project attributes, and calibration factors [6], [14], [15], [18], [34], [36].

4.8.4 Dependency Coherence

Dependency coherence measures the structural soundness of the generated dependency graph:

$$\boldsymbol{C_{\text{dep}}(Y) = max(0,\ 1 - 0.1W)}$$

where W is the number of structural warnings produced by graph validation. These warnings may arise from cycles, suspicious isolated tasks, or implausible dependency direction. This score therefore reflects whether the task graph is execution-ready. The use of graph-based dependency reasoning is consistent with project scheduling and critical-path planning methods [33], and its implementation can be supported by graph-analysis libraries such as NetworkX [21].

4.8.5 Supplementary Evaluation Metrics

In addition to the main four-part score, the framework employs complementary metrics for more detailed empirical assessment.

For estimation quality, it uses MMRE and PRED(25), which are commonly used in software effort estimation studies and surveys [15], [18], [36]:

$$MMRE == \frac{1}{n}\sum_{i=1}^{n}\frac{|A_i - P_i|}{A_i}$$

$$\boldsymbol{PRED(25) == \frac{1}{n}\sum_{i=1}^{n} b1\left(\frac{|A_i - P_i|}{A_i} \leq 0.25\right)}$$

where $A_i$is the actual effort and $P_i$is the predicted effort.

For requirement-type performance, it computes per-class F1 scores for FR and NFR classification whenever ground-truth task labels are available. This supports more detailed analysis of how reliably the system separates different requirement categories [38].

At the experimental-report level, the system also tracks:

fallback rate,

average critic score,

average overall planning score.

The current operational decision thresholds are:

fallback rate $\leq \mathbf{30}\%$,

average critic score $\geq \mathbf{0.85}$,

average overall planning score $\geq \mathbf{0.78}$.

These thresholds serve as practical acceptance criteria for judging whether a given planning mode is reliable enough for routine use. Since these thresholds are implementation-specific, they are treated as internal experimental criteria rather than universal values.

4.8.6 Penalty Scheme

The proposed framework employs explicit penalty schemes at two levels: plan-quality validation and risk scoring. Penalty-based assessment is suitable in this context because planning errors differ in severity; for example, a cyclic dependency is more serious than a minor warning in task wording. This approach supports interpretable validation and aligns with project management practices that emphasize structured risk and quality assessment [37].

4.8.7 Critic Penalty Function

The Critic Agent computes a plan-quality score by starting from full credit and subtracting penalties according to issue severity:

$$\boldsymbol{Scritic}(\boldsymbol{Y}) = max\left(\mathbf{0},\ \mathbf{1} - \boldsymbol{j} = \mathbf{1}\sum \boldsymbol{m\pi}_{\mathrm{j}}\right)$$

where each $\boldsymbol{\pi}_{\mathrm{j}}$ is determined by the issue severity:

error:$0.\mathbf{20}$

warning: $\mathbf{0.05}$

info: $\mathbf{0.01}$

Thus, the critic score is not a learned probability, but a deterministic confidence measure derived from structural and logical violations. Based on this score, the plan is assigned one of three statuses:

rejected if any hard error exists or if $S_{critic} < 0.50$

needs_revision if.$\mathbf{0.50} \leq \boldsymbol{S_{critic}} < \mathbf{0.80}$

approved if $\boldsymbol{S_{critic}} \geq \mathbf{0.80}$

This makes the critic stage an explicit acceptance barrier rather than a passive advisory layer. The critic role is also consistent with multi-agent LLM architectures in which separate agents perform generation, review, critique, and refinement functions [4], [16], [17].

4.8.8 Risk Penalty Function

The Risk Analyzer uses a separate severity-accumulation penalty function:

$$\boldsymbol{Srisk}(\boldsymbol{Y}) = min\left(\mathbf{1},\ \boldsymbol{i} = \mathbf{1}\sum \boldsymbol{n\rho i}\right)$$

with severity penalties defined as:

critical: $\mathbf{0.30}$

high: $\mathbf{0.15}$

medium: $\mathbf{0.05}$

low: $\mathbf{0.01}$

The resulting score is then mapped to qualitative project risk levels:

critical if $\boldsymbol{S_{risk} \geq 0.70}$

high if $\boldsymbol{S_{risk} \geq 0.40}$

medium if $\boldsymbol{S_{risk} \geq 0.20}$

low otherwise

This penalty model allows heterogeneous risk observations to be aggregated into a single interpretable project-level signal. This is consistent with the broader project management view that project risks should be identified, analyzed, prioritized, and translated into mitigation decisions [37].

4.8.9 Adopted Inference Algorithms

The proposed system adopts a hybrid inference strategy rather than a single algorithmic mechanism. This is one of the key design decisions of the architecture. Hybridization is necessary because project planning requires both semantic reasoning and deterministic control: LLMs can interpret ambiguous language, while rule-based methods provide safety, reproducibility, and fallback behavior [3], [4], [7], [8].

4.8.9.1 LLM-Guided Structured Planning Inference

The primary inference mechanism is prompt-based structured generation using a local LLM deployed through Ollama [19]. The model receives:

the original input text,

the normalized requirements,

atomic mapping constraints,

FR/NFR classification rules,

complexity rules,

dependency rules,

and optional retrieved knowledge-base context.

The LLM is required to return a strict JSON object containing the planned tasks. This inference mode is therefore not free-form generation; it is schema-constrained structured planning inference. The use of transformer-based LLMs and prompting-based reasoning is grounded in recent advances in language modeling, few-shot learning, and chain-of-thought-style reasoning [1], [31], [32].

4.8.9.2 Compact-Retry and Repair Inference

If the first LLM attempt fails because of malformed JSON, missing wrappers, or partial structural corruption, the planner launches a compact retry using a shorter prompt. If the result remains structurally weak but repairable, the system runs a quality-repair inference pass in which the model is explicitly instructed to correct mismatched fields, dependency problems, and cardinality violations.

Thus, the inference pipeline is not a single LLM call, but a controlled cascade:

primary generation,

compact retry if necessary,

repair prompt if quality issues are repairable,

deterministic fallback if failure persists.

This controlled generation-and-repair strategy supports the broader goal of making LLM outputs usable in software engineering workflows, where outputs must satisfy structural and logical constraints rather than merely appear fluent [3], [4], [8].

4.8.9.3 . Rule-Based Fallback Inference

When LLM inference fails or is disabled, the system switches to a deterministic rule-based inference algorithm. In this mode, each requirement is transformed into a task using explicit heuristic functions:

requirement type inference through lexical and semantic pattern rules,

action-title synthesis through verb-object normalization,

complexity scoring through rule-based complexity classification,

direct dependency inference through workflow and semantic-precedence heuristics.

This fallback path ensures that the system remains operational even in the absence of generative inference. The use of fallback logic is important because software planning systems must remain reliable under uncertainty, incomplete inputs, or model failure conditions [37], [38].

4.8.9.4 . Rule-Based Complexity Inference

Task complexity is inferred through a deterministic heuristic function. In the implemented logic:

read-only operations are typically mapped to complexity 1,

simple CRUD or transactional operations to complexity 2,

authentication, security, localization, and multi-step logic to complexity 3,

external integration, RBAC, performance constraints, and offline architectural constraints to complexity 4,

highly distributed or cross-system orchestration is reserved for complexity 5.

This rule-based complexity inference provides a stable baseline that is later used both for validation and for estimation. This is consistent with software estimation models that rely on complexity, size, technical factors, environmental factors, and calibration parameters to derive effort estimates [14], [15], [18], [34], [36].

4.8.9.5 . Retrieval-Augmented Inference

The system further adopts retrieval-augmented inference through a vector knowledge base. Query text is embedded and compared against stored planning records in semantic space, and the most relevant examples are returned for prompt augmentation. This allows the planner to reason with contextual support rather than relying exclusively on parametric model memory.

This design follows the principle of retrieval-augmented generation, in which external knowledge is retrieved and inserted into the generation process to improve grounding and reduce unsupported generation [2]. The implementation is supported by ChromaDB as the vector database [20] and Sentence Transformers for dense embedding generation [25].

4.8.9.6 Estimation Inference by Distance-Weighted Retrieval

For effort estimation, the framework uses a hybrid rule-plus-retrieval algorithm. First, a rule-based estimate H_{rule} is computed from complexity and requirement type. Then a retrieved estimate $\boldsymbol{H_{rag}}$ is derived from similar records using inverse-distance weighting:

$$H_{rag} = \frac{\sum_{i=1}^{n} h_i w_i}{\sum_{i=1}^{n} w_i}, \quad w_i = \frac{1}{\max(d_i, \epsilon)}$$

where $\boldsymbol{hi}$ is the historical effort value and $\boldsymbol{di}$ is the semantic distance of the retrieved record.

The final estimate is a bounded blend:

$$\boldsymbol{H_{final} = \alpha H_{rag} + (1 - \alpha) H_{rule}}$$

where:

$\boldsymbol{\alpha = 0.35}$ for low-complexity tasks,

$\boldsymbol{\alpha = 0.15}$ for higher-complexity tasks.

This choice ensures that retrieval calibrates the rule-based estimate rather than replacing it completely. The overall estimation approach is consistent with software effort estimation research, where historical data, calibrated models, and project-specific factors are used to improve prediction quality [6], [14], [15], [18], [34], [36].

4.8.9.7 Dependency Inference

Dependency inference is also hybrid and largely heuristic. The planner infers direct predecessors based on:

workflow order,

role and access preconditions,

artifact-consumer versus data-producer relations,

foundational NFR constraints,

and semantic overlap between earlier and later tasks.

Additional propagation rules refine the task chain, including:

workflow-chain completion,

NFR constraint propagation,

correction of inverted performance dependencies.

This makes dependency inference a structured reasoning task rather than a naive adjacency assignment. The dependency structure is important because project schedules depend on precedence relationships, execution constraints, and critical-path behavior [33], [37].

4.8.9.8 Graph Inference

After task inference, the dependency structure is converted into a directed graph, and critical-path inference is performed through the longest weighted DAG path. This enables the system to derive execution order, bottlenecks, parallel groups, and schedule-sensitive tasks from the inferred plan. This graph-based reasoning is supported by graph analysis principles and practical graph-processing tools such as NetworkX [21], while critical-path reasoning is rooted in classical project scheduling methods [33].

4.8.10 Methodological Significance

The combination of these evaluation functions, penalty schemes, and inference algorithms gives AI PM its hybrid character. The system does not rely solely on language generation, nor does it reduce planning to rigid deterministic rules. Instead, it integrates:

structured LLM reasoning,

retrieval-augmented contextual support,

rule-based fallback inference,

penalty-driven validation,

and multi-metric evaluation.

This methodological design is aligned with recent directions in LLM-assisted software engineering, retrieval-augmented generation, multi-agent reasoning, and structured requirements processing [2], [3], [4], [7], [8], [16], [17], [38].

4.9 Knowledge Sources, Datasets, and Preprocessing

4.9.1 Vector Knowledge Base, Sample Briefs, and Ground Truth

The proposed AI PM framework relies on a deliberately heterogeneous data foundation designed to support both runtime planning and empirical evaluation. Rather than depending on a single monolithic dataset, the system integrates three main knowledge sources: a vector knowledge base (KB) for retrieval-augmented reasoning and estimation support, a set of curated sample briefs representing diverse software domains, and a ground-truth benchmark file that defines expected planning behavior for evaluation. This combination enables the system to operate as both an intelligent planning tool and a reproducible research artifact. The use of retrieval-augmented knowledge support is consistent with retrieval-augmented generation approaches, where external knowledge is retrieved to ground model outputs and improve reasoning reliability [2].

4.9.2 Vector Knowledge Base

A central knowledge source in the architecture is the vector knowledge base, which is implemented as a persistent ChromaDB collection and embedded using the SentenceTransformer all-MiniLM-L6-v2 model [20], [25]. The KB stores semantically retrievable planning and estimation records rather than raw documents only. In the current project implementation, the seeded knowledge base contains 100 curated documents, distributed across the following categories:

30 historical records, representing project-level effort references inspired by benchmark-style historical software projects and synthetic planning cases.

10 COCOMO-style calibration points, used to provide coarse effort calibration signals grounded in classical estimation logic [27], [34].

38 task-level patterns, representing reusable implementation and estimation examples for common software engineering tasks.

14 planning examples, used primarily to guide decomposition, FR/NFR classification, and planning structure.

8 estimation-pattern records, used to calibrate task effort more precisely during the estimation stage.

The KB spans multiple software domains, including fintech, education, IoT, CRM, e-commerce, healthcare, library systems, social platforms, and SaaS billing, among others. This domain diversity is methodologically important because the project is intended as a general-purpose requirements-to-plan framework, not a domain-specific planner.

From a preprocessing perspective, each KB record is transformed into:

a unique document identifier,

a normalized text body,

structured metadata such as category, domain, effort values, and team size where applicable,

a dense semantic embedding.

The metadata layer is preserved alongside the embeddings so that retrieval is not limited to semantic distance alone; instead, semantic similarity can later be interpreted in light of category and domain information. This allows the KB to function as a controlled external memory for both planning augmentation and effort calibration. This design follows the general principle of retrieval-augmented generation, while the use of dense sentence embeddings supports semantic similarity retrieval over textual planning records [2], [25].

4.9.3 Sample Briefs

The second major source of data consists of curated sample software project briefs. In the raw document corpus, the project includes 11 primary project briefs covering domains such as:

- CRM,e-commerce,
- fintech,
- HR systems,
- IoT dashboards,
- library management,
- learning management systems,
- SaaS billing,
- social platforms,
- and generalized clinic or academic portal scenarios.

These briefs are intentionally heterogeneous in scope, actor structure, and constraint style. Some emphasize transactional workflows, others focus on integration-heavy systems, while others contain strong non-functional requirements such as performance, encryption, availability, localization, or compliance. This diversity is essential because it exposes the planning pipeline to a broad range of software-planning conditions rather than a narrowly homogeneous task family. The inclusion of functional and non-functional concerns is aligned with requirements engineering standards that distinguish system behavior, quality attributes, constraints, and stakeholder needs [38].

In addition to the main narrative briefs, the project also includes:

a structured requirements template sample,

a single-requirement edge-case sample,

several inline benchmark prompts embedded directly in the evaluation set.

This provides a useful mix of full-length briefs, template-based requirements, and minimal cases for robustness testing.

4.9.4 Ground Truth Benchmark

The third and most important evaluation dataset is the ground-truth benchmark file, which contains 20 benchmark samples. Of these:

19 samples use the brief input format,

1 sample uses the template format,

14 samples are file-backed,

6 samples are inline textual inputs.

The ground truth in this project is not defined as a single exact gold task list. Instead, it follows a constraint-oriented benchmark design, which is more appropriate for planning systems whose outputs may vary lexically while still remaining semantically correct. Each benchmark sample therefore specifies expected properties such as:

minimum and maximum task count,

minimum FR and NFR counts,

minimum requirement coverage,

expected FR/NFR distribution,

anchor task classifications through title fragments,

optionality expectations,

dependency-chain expectations,

critical-path constraints,

and reference task-effort values for MMRE and PRED(25) computation [15], [18], [36].

This is an important methodological decision. Exact string-level ground truth would be overly brittle for a structured planning system driven by hybrid reasoning. By contrast, the chosen benchmark design evaluates whether the generated plan satisfies the functional, structural, and estimation-related intent of the sample rather than merely reproducing a predetermined wording. This is consistent with the nature of requirements engineering evaluation, where semantic coverage, traceability, classification quality, and structural correctness are more important than surface-level textual matching [7], [8], [38].

4.9.5 Preprocessing of Narrative Briefs

For narrative project briefs, preprocessing is performed by a dedicated Brief Parser. This parser first splits the document into recognized sections such as:

project title,

overview,

main features,

functional requirements,

non-functional requirements,

actors,

target users,

expected benefits,

and constraints or special notes.

It then extracts bullet-based requirements where available and falls back to sentence segmentation when needed. Several normalization operations are applied during this stage:

cleaning and trimming of requirement text,

extraction of actor information,

splitting of compound quality constraints into atomic requirement items,

detection and normalization of non-functional requirement concerns,

exclusion of explicit out-of-scope statements,

conversion of loosely phrased benefits or constraints into requirement-like statements where appropriate.

A particularly important preprocessing step is the treatment of compound NFR statements. For example, requirements referring jointly to encryption at rest and in transit, or availability and failover, may be expanded into more atomic and normalized forms so that later planning stages can reason over them consistently. This preprocessing approach supports the general requirements engineering goal of transforming informal stakeholder input into structured, analyzable, and traceable requirement units [38]. It is also consistent with recent LLM-supported requirements engineering research that focuses on structuring and generating requirements from natural-language input [7], [8].

4.9.6 Preprocessing of Template-Based Requirements

For template-formatted input, preprocessing is handled by a dedicated Template Parser. This parser identifies requirement blocks of the form [REQ-XX] and validates their fields. In the current implementation, it verifies:

the presence of a valid description,

valid requirement type, Functional or Non-Functional,

valid priority values, High, Medium, Low,

and a minimum descriptive richness threshold.

After validation, each block is transformed into a normalized RequirementItem while preserving its source identifier. Optional fields such as actor and notes are folded into the textual representation so that the downstream planner receives a unified requirement string without losing contextual information. This preserves traceability between source requirements and derived planning artifacts, which is a central principle in requirements engineering practice [38].

4.9.7 Shared Planner-Side Preprocessing

Beyond the parsers themselves, the planning pipeline performs additional shared preprocessing before inference. This includes:

source-preserving normalization of requirement text,

deduplication of near-duplicate requirements using similarity-aware logic,

atomic decomposition of compound requirements when decomposition mode is enabled,

semantic classification hints for FR/NFR interpretation,

and complexity priors derived from rule-based heuristics.

As a result, the planning model does not receive raw, noisy input directly. Instead, it receives a normalized, traceable, and partially structured requirement set that substantially improves inference stability. This preprocessing stage supports controlled LLM-assisted planning because structured and traceable inputs reduce ambiguity and improve the reliability of downstream generation and validation [3], [7], [8], [38].

4.9.8 Methodological Importance of the Data Design

The combination of the KB, sample briefs, and ground-truth benchmark provides the project with an unusually strong methodological foundation. The KB supports runtime intelligence, the sample briefs provide domain diversity, and the ground truth enables measurable evaluation. Together, they allow the proposed system to be studied not merely as an implementation prototype, but as a reproducible hybrid planning framework grounded in both operational knowledge and benchmark-driven validation. This design is consistent with the broader methodological need to evaluate software planning and estimation systems using explicit criteria, historical or benchmark-style references, and measurable planning outputs [6], [15], [18], [36], [37].

4.10 Chapter Conclusion and Research Contribution

This chapter has presented the proposed AI PM architecture as a hybrid multi-agent planning framework designed to transform natural-language software requirements into structured, validated, and execution-oriented planning artifacts. The discussion moved from the high-level architectural concept to the detailed specification of its components, the internal pipeline logic, the evaluation and penalty functions, the adopted inference algorithms, and the knowledge and benchmark resources that support both execution and empirical assessment.

The chapter makes an important research contribution by demonstrating that intelligent software project planning should not be treated as a single-step generation problem. Instead, the findings of this chapter support a different architectural position: reliable planning emerges from the integration of structured parsing, LLM-guided reasoning, retrieval-augmented contextual grounding, deterministic validation, graph-based dependency modeling, effort-aware enrichment, and multi-metric evaluation. In this sense, the contribution of the chapter is both architectural and methodological. This position is aligned with recent directions in LLM-assisted software engineering, multi-agent LLM systems, retrieval-augmented generation, and structured requirements processing [2], [3], [4], [7], [8], [16], [17].

From an architectural perspective, the chapter contributes a concrete design for a local-first, hybrid, role-specialized planning system that balances generative flexibility with deterministic control. From a methodological perspective, it contributes a reproducible evaluation setting in which planning quality is assessed through explicit benchmark constraints, critic scoring, dependency coherence, and estimation-oriented metrics rather than through vague qualitative impressions alone. From a data perspective, it contributes a structured integration of a vector knowledge base, curated project briefs, and a constraint-based ground-truth benchmark that collectively enable both runtime support and scientific evaluation.

Accordingly, this chapter establishes the core scientific basis of the proposed system and clarifies how the research moves beyond both traditional planning methods and narrowly scoped AI prototypes. It shows that the novelty of the proposed work lies not in the isolated use of LLMs, but in the construction of a trustworthy, auditable, and evaluation-ready hybrid planning pipeline. The next chapter therefore builds naturally upon this foundation by presenting the implementation details and operational realization of the proposed framework.

5 Chapter five: Eexperimental Evaluation

5.1 Chapter Introduction

This chapter presents the experimental framework used to evaluate the proposed AI PM system. While the previous chapter established the architectural and methodological foundations of the framework, the current chapter focuses on its empirical assessment under controlled benchmark conditions. The evaluation is designed to determine whether the system can transform natural-language software requirements into planning artifacts that are not only structurally valid, but also semantically complete, quantitatively reasonable, and operationally useful. This evaluation direction is aligned with requirements engineering principles that emphasize traceability, validation, and structured analysis of requirements-derived artifacts [38].

To achieve this objective, the chapter defines the software environment in which the system was executed, the benchmark resources used during experimentation, and the evaluation criteria applied to the generated outputs. Because software project planning is a multidimensional task, the assessment does not depend on a single metric. Instead, the evaluation framework combines structural metrics, quantitative performance metrics, and an aggregate overall score in order to assess coverage, dependency quality, classification behavior, estimation quality, and complexity realism. The chapter also documents the execution protocol applied to the 20 benchmark samples used in the study, thereby ensuring methodological transparency and reproducibility. The use of multiple evaluation criteria is consistent with software effort estimation and project management literature, where planning quality cannot be reduced to a single numerical indicator [15], [18], [36], [37].

5.2 Experimental Environment and Software Tools Used

Ollama, ChromaDB, NetworkX, FastAPI, React

The experimental implementation of AI PM was conducted in a local-first software environment that combines Python-based backend services with a modern web frontend and local AI inference tooling. This environment was selected deliberately to support privacy-preserving execution, reproducibility, and low-cost deployment without dependence on external cloud inference services.

At the reasoning layer, the system uses Ollama as the local model-serving framework [19]. Ollama hosts the planning model and exposes it through a local HTTP interface, allowing the Planner Agent and auxiliary reasoning layers to request structured JSON outputs without transmitting project requirements to remote services. This choice is particularly important because the project aims to demonstrate that intelligent software planning can be implemented in a self-contained local environment.

For retrieval-augmented reasoning, the system uses ChromaDB as a persistent vector database [20]. The seeded knowledge base stores planning examples, estimation patterns, and curated historical effort records. Semantic retrieval is supported through dense embeddings generated by the SentenceTransformer all-MiniLM-L6-v2 model [25]. The KB is persisted locally and is activated in knowledge-grounded planning modes such as rules_kb and llm_kb.

This design follows the principle of retrieval-augmented generation, where retrieved external knowledge is used to ground model behavior and improve output reliability [2].

Graph-level execution analysis is implemented using NetworkX, which models the task plan as a directed graph and computes structural analytics such as bottlenecks, root tasks, leaf tasks, parallel groups, and the critical path [21]. This graph engine plays a major role in both planning interpretation and dependency-quality evaluation. The use of graph-based dependency analysis is also connected to classical project scheduling and critical-path reasoning [33].

The backend service layer is implemented using FastAPI, which exposes the project pipeline through API routes for planning, monitoring, evaluation, export, and auxiliary AI explanation [22]. FastAPI was chosen because it provides a lightweight yet high-performance framework for orchestrating the pipeline components and serving structured planning outputs through a clean API interface.

The frontend is implemented using React with Vite as the build tool [23], [24]. In the current implementation, the frontend stack includes React 19, React Router, and modern TypeScript-based tooling. Although the benchmark evaluation itself is executed primarily through the Python evaluation pipeline, the React interface provides practical support for inspecting generated tasks, graph artifacts, summaries, and progress signals.

In addition to these principal technologies, the environment also relies on several supporting libraries, including:

Pydantic for schema validation,

Requests for local LLM communication,

Sentence-Transformers for semantic embeddings [25],

Uvicorn for API serving,

and Pytest for backend verification.

Taken together, this environment reflects the project’s core design philosophy: a hybrid, modular, locally executable, and experimentally auditable AI planning system.

5.3 Structural Evaluation Metrics: Coverage, Dependency Coherence, and Critic Score

Because the output of AI PM is a structured project plan rather than a simple classification label, the first category of evaluation focuses on structural quality. These metrics assess whether the produced plan is complete, coherent, and valid from a planning perspective. This is consistent with requirements engineering principles, where generated artifacts should be evaluated in terms of completeness, consistency, traceability, and validity [38].

Requirement Coverage measures the extent to which the generated task plan reflects the source requirements. It is defined as:

$$\boldsymbol{Coverage} = \frac{\textbf{covered requirements}}{\textbf{total requirements}}$$

A requirement is considered covered when at least one generated task preserves its corresponding source reference and planning intent. This metric is important because a structurally correct plan is still inadequate if it omits major portions of the input specification. The emphasis on coverage and traceability is aligned with requirements engineering standards [38].

Dependency Coherence measures the structural validity of the dependency graph derived from the task plan. In the implemented system, graph validity is assessed using DAG-based validation and warning generation. The normalized score is defined as:

$$\boldsymbol{DependencyCoherence = max\,(0,\ 1 - 0.1W)}$$

where $\boldsymbol{W}$ denotes the number of dependency-related warnings. If the graph contains cycles or suspicious structural anomalies, the warning count increases and the score decreases. This metric is especially significant because dependency quality directly affects execution ordering, critical-path analysis, and sprint planning. The use of dependency graphs and critical-path reasoning is supported by project scheduling literature and graph-based analysis tools [21], [33].

Critic Score is the structured quality score produced by the Critic Agent after reviewing the generated task plan. The critic penalizes plan issues according to severity:

$$\boldsymbol{CriticScore = max\big(0,\ 1 - (0.20E + 0.05W + 0.01I)\big)}$$

where:

$\boldsymbol{E}$ is the number of critic errors,

$\boldsymbol{W}$ is the number of warnings,

$\boldsymbol{I}$ is the number of informational issues.

This metric provides a deterministic and interpretable assessment of plan quality. It captures logical inconsistencies such as invalid dependencies, missing estimates, implausible FR/NFR ratios, and structurally weak task groupings. Unlike the overall score discussed later, the Critic Score functions as a quality gate, since a sufficiently poor score or any hard error may lead to plan rejection. This critic-style validation role is consistent with multi-agent LLM architectures where separate agents may generate, review, critique, or refine outputs [4], [16], [17].

Together, these three structural metrics evaluate whether the generated plan is complete, graph-consistent, and acceptable under formal review.

5.4 Quantitative Evaluation Metrics: MMRE, PRED(25), F1-FR, F1-NFR, and Complexity Entropy

In addition to structural validity, the proposed system is evaluated using a set of quantitative metrics that assess estimation quality, classification behavior, and complexity realism.

MMRE, Mean Magnitude of Relative Error, is used to evaluate task-effort estimation quality. MMRE is widely used in software effort estimation studies and surveys [15], [18], [36]:

$$\boldsymbol{MMRE} = \frac{\mathbf{1}}{\boldsymbol{n}}\sum_{\boldsymbol{i=1}}^{\boldsymbol{n}} 1\left(\frac{|\boldsymbol{A_i} - \boldsymbol{P_i}|}{\boldsymbol{A_i}} \leq 0.25\right)$$

where $\boldsymbol{A}_{\mathrm{i}}$ is the actual effort for task iii and p_i is the predicted effort. Lower MMRE values indicate better estimation accuracy.

PRED(25) complements MMRE by measuring the proportion of predictions whose relative error does not exceed 25%:

$$\boldsymbol{PRED(25)} = \frac{\mathbf{1}}{\boldsymbol{n}}\sum_{\boldsymbol{i=1}}^{\boldsymbol{n}} \mathbf{1}\left(\frac{|\boldsymbol{A_i} - \boldsymbol{P_i}|}{\boldsymbol{A_i}} \leq \mathbf{0.25}\right)$$

This metric is useful because MMRE alone may hide the proportion of practically acceptable estimates. It is also commonly used alongside MMRE in software effort estimation evaluation [15], [18], [36].

F1-FR and F1-NFR evaluate the system's ability to classify generated tasks as functional or non-functional in alignment with benchmark expectations. These per-class F1 scores are computed from precision and recall:

$$\boldsymbol{F1} = \frac{\boldsymbol{2PR}}{\boldsymbol{P + R}}$$

where $\boldsymbol{P}$ is precision and $\boldsymbol{R}$ is recall. In the project benchmark, these scores are computed by matching expected task-title fragments from the ground truth against generated tasks and comparing the resulting FR/NFR labels. The use of separate F1 values for FR and NFR is methodologically important because errors in non-functional classification can significantly affect later effort, dependency, and risk reasoning. This distinction is aligned with requirements engineering practice, where functional requirements and non-functional requirements represent different forms of system expectations and constraints [38].

Complexity Entropy measures how realistically the system distributes tasks across complexity levels. Rather than rewarding a narrow or uniform complexity assignment, this metric evaluates the normalized Shannon entropy of the complexity distribution:

$$\boldsymbol{ComplexityEntropy} = \frac{-\sum_{i=1}^{k} \boldsymbol{p_i} \log_2 \boldsymbol{p_i}}{\log_2 \mathbf{5}}$$

where $\boldsymbol{pip\_ipi}$ is the proportion of tasks assigned to complexity class iii. The denominator normalizes the metric to the interval [0,1][0,1][0,1] across five supported complexity levels. Higher values indicate a richer and more balanced complexity distribution, while very low values may suggest over-uniform planning behavior. The motivation for evaluating task complexity is consistent with software effort estimation research, where complexity, size, technical factors, and calibration parameters are central to effort prediction [6], [14], [15], [18], [34], [36].

These quantitative metrics provide an empirical view of how well the system estimates effort, preserves requirement types, and generates realistic task distributions.

5.5 The Overall Evaluation Criterion: Overall Score and the Rationale for Its Selection

Because no single metric can adequately represent planning quality, the study adopts an aggregate normalized evaluation criterion called the Overall Score. In the current implementation, this score combines four planning-quality dimensions:

$$\boldsymbol{OverallScore} = \frac{\boldsymbol{Coverage + ClassificationConsistency + ComplexityEntropy + DependencyCoherence}}{\mathbf{4}}$$

The Classification Consistency term measures how closely the generated FR/NFR distribution matches the expected benchmark distribution:

$$\boldsymbol{ClassificationConsistency} = \max\left(\mathbf{0},\ 1 - \frac{\left|\boldsymbol{FR_{actual}} - \boldsymbol{FR_{expected}}\right|}{\mathbf{0.3}}\right)$$

The rationale for selecting this aggregate formulation is fourfold.

First, each component is normalized to the range $[\mathbf{0}, \mathbf{1}]$, which makes direct averaging both mathematically simple and interpretively clear.

Second, the four selected terms correspond to the main planning objectives of the system:

Coverage reflects semantic completeness.

Classification Consistency reflects fidelity to requirement type balance.

Complexity Entropy reflects realism of workload calibration.

Dependency Coherence reflects structural executability.

Third, the selected formulation avoids over-reliance on effort-estimation metrics alone. Although MMRE and PRED(25) are important, they measure only one aspect of planning quality and depend on the availability of comparable actual-hours references. They are therefore treated as supplementary quantitative metrics rather than the sole basis for overall system quality. This is consistent with prior software effort estimation literature, where estimation accuracy is important but does not fully represent overall project planning quality [15], [18], [36], [37].

Fourth, the Critic Score is intentionally not merged into the Overall Score. This is because the critic already functions as an explicit quality gate with its own penalty model; incorporating it directly into the same aggregate score would risk double-counting structural errors that are already reflected in coverage, classification, and dependency behavior.

Accordingly, the Overall Score was selected because it offers a balanced, interpretable, and architecture-aligned summary measure that reflects the multi-dimensional nature of software project planning.

5.6 Experimental Execution Protocol on the 20 Benchmark Samples

The experimental protocol of this study was designed to evaluate the proposed framework systematically across a benchmark dataset consisting of 20 samples. The benchmark includes:

19 brief-format samples,

1 template-format sample,

14 file-backed inputs,

and 6 inline textual inputs.

The protocol was executed through the project's Python-based evaluation pipeline and follows the steps below.

5.6.1 Benchmark loading

The benchmark dataset is loaded from ground_truth.json, where each sample defines the input source together with its expected planning properties, including task-count bounds, classification expectations, coverage thresholds, and reference effort values where available. This constraint-oriented benchmark structure is suitable for evaluating requirements-to-plan systems because generated outputs may differ lexically while still satisfying the same semantic and structural planning objectives [7], [8], [38].

5.6.2 Mode selection

The evaluation framework supports multiple execution modes:

rules for rule-based planning only,

rules_kb for rule-based planning with knowledge-base calibration,

llm for LLM planning without KB grounding,

llm_kb for the full hybrid pipeline,

zero_shot for a lightweight LLM baseline outside the full pipeline.

In methodological terms, the primary experimental condition is llm_kb, since it corresponds to the complete AI PM architecture. The remaining modes are used for comparison and ablation analysis. This type of comparative evaluation is useful for isolating the contribution of retrieval support, LLM reasoning, and rule-based fallback logic [2], [3], [4], [16], [17].

5.6.3 Input parsing and normalization

Each sample is parsed according to its declared format. Brief inputs are normalized through the Brief Parser, whereas structured template inputs are processed by the Template Parser. When applicable, inline text is converted into normalized requirement items. This preprocessing step supports the transformation of informal natural-language input into structured and traceable requirement representations [7], [8], [38].

5.6.4 Planning execution

The planner generates a task plan according to the selected mode. In llm_kb, the system uses local LLM inference with retrieval-augmented context. If the LLM fails or produces unacceptable output, fallback behavior is recorded explicitly. This hybrid execution strategy combines LLM-guided reasoning, retrieval-augmented grounding, and deterministic fallback logic [2], [3], [4], [16], [17].

5.6.5 Estimation, critique, and graph analysis

The generated plan is enriched with effort metadata, reviewed by the Critic Agent, and converted into a dependency graph. This produces the task plan, critic output, and structural analytics required for scoring. These stages are connected to software effort estimation, multi-agent review, and graph-based project scheduling methods [4], [14], [15], [18], [21], [33], [36].

5.6.6 Metric computation

For each sample, the pipeline computes structural and quantitative metrics, including coverage, dependency coherence, critic score, MMRE, PRED(25), F1-FR, F1-NFR, and complexity entropy. The Overall Score is then computed from the normalized planning dimensions. These metrics allow the evaluation to capture requirements coverage, classification behavior, estimation quality, and dependency validity rather than relying on a single success criterion [15], [18], [36], [38].

5.6.7 Aggregation and reporting

After all 20 samples have been processed, the system aggregates the results into summary statistics such as:

pass rate,

average overall score,

average critic score,

fallback rate,

direct-generation rate,

and average task count.

The results are then written to machine-readable and human-readable reports such as evaluation_report.json, evaluation_report.md, and, when applicable, ablation_report.json. This reporting design supports reproducibility, auditability, and transparent comparison across experimental modes [37], [38].

5.6.8 Comparative ablation protocol

For deeper experimental analysis, the system can also run an ablation study across multiple conditions, including Rules only, Rules + KB, LLM + KB, and Zero-shot LLM. In this protocol, the system additionally reports comparative deltas and bootstrap confidence intervals for selected quantitative metrics such as MMRE and PRED(25). This ablation-oriented comparison helps determine whether the complete hybrid configuration provides measurable benefits over simpler planning modes [2], [3], [4], [16], [17].

This protocol ensures that every benchmark sample is processed through the same logical stages while still allowing controlled comparison between hybrid, rule-based, and baseline conditions. As such, it provides a reproducible and methodologically defensible basis for evaluating the proposed AI PM framework.

5.7 Results of the Proposed Model

This section presents the empirical results of the proposed **AI PM** model under the primary experimental condition, namely the **full hybrid pipeline (`llm_kb`)**, evaluated on the **20-sample benchmark dataset**. The results indicate that the proposed model achieved **complete benchmark pass coverage**, while also producing strong structural quality indicators and a high critic-based acceptance score. At the same time, the findings reveal that the current local LLM layer still relies heavily on fallback execution, which is an important observation for interpreting the practical maturity of the system. This evaluation approach is consistent with requirements engineering principles that emphasize traceability, validation, and structured assessment of derived artifacts [38], as well as with software planning literature that treats planning quality as a multidimensional problem rather than a single-metric outcome [15], [18], [36], [37].

5.7.1 Overall Results

5.7.1.1 Success Rate, Means, and Standard Deviations

Across the 20 benchmark samples, the proposed model achieved a **100.0% pass rate** and a **100.0% successful run rate**, meaning that every sample produced a valid final planning output. However, only **25.0%** of the samples were accepted through direct LLM generation, while **75.0%** required fallback to the rule-based planning path. This result demonstrates that the proposed framework is operationally robust, but that its current generative layer remains partially dependent on deterministic recovery mechanisms. This supports the methodological value of hybrid planning architectures that combine LLM-based reasoning with rule-based control and validation [3], [4], [7], [8], [16], [17].

The mean results and standard deviations computed across the 20 benchmark cases are summarized below.

Metric	Mean	Standard Deviation
Overall Score	0.770	0.135
Coverage	1.000	0.000
Classification Consistency	0.613	0.360
Complexity Entropy	0.476	0.211
Dependency Coherence	0.990	0.030
Critic Score	0.986	0.034
MMRE	0.301	0.178
PRED(25)	0.691	0.215
F1-FR	0.573	0.298
F1-NFR	0.482	0.430
Task Count	7.95	4.38

These findings support several important observations. First, **coverage was perfect across all samples**, indicating that the pipeline consistently preserved requirement traceability and did not omit benchmark requirements at the final accepted-plan level. The emphasis on traceability and requirement coverage is aligned with requirements engineering standards [38]. Second, **dependency coherence was very high** ((0.990)), which confirms that the graph-construction and dependency-repair logic produced structurally executable plans in nearly all cases. This is consistent with the importance of dependency modeling and critical-path reasoning in project planning [21], [33], [37]. Third, the **Critic Score averaged 0.986**, showing that the final accepted outputs were of high structural quality once the planner and fallback mechanisms completed their work. The use of a dedicated review or critic layer is aligned with multi-agent LLM designs in which specialized agents are used for generation, validation, and refinement [4], [16], [17].

At the same time, the quantitative metrics reveal more moderate performance in some dimensions. In particular, **classification consistency** and **F1-NFR** show greater variability, suggesting that non-functional reasoning remains less stable than functional decomposition. This is important because requirements engineering distinguishes functional requirements from non-functional requirements and treats both as essential but different categories of system specification [38]. Similarly, the estimation metrics show usable but not uniformly strong performance, with **MMRE = 0.301** and **PRED(25) = 0.691**. These metrics are commonly used in software effort estimation evaluation [15], [18], [36]. This indicates that the current version of the system is stronger in **structural planning quality** than in **fully stable effort prediction** across all benchmark domains.

A comparative reading against the rule-only baseline is also informative. While the proposed `llm_kb` system improved the **average overall score** from **0.749** to **0.770**, and raised both **classification consistency** and **dependency coherence**, its **effort-estimation metrics**

remained slightly less stable than the deterministic baseline. This suggests that the present value of the hybrid model lies primarily in **planning expressiveness and structural quality**, while effort estimation still benefits strongly from rule-based regularization. This observation is consistent with software effort estimation literature, where estimation performance often depends on calibration, stable historical references, and controlled model assumptions [6], [14], [15], [18], [34], [36].

5.7.2 Assessment of the Declared Research Objectives in Chapter One

The experimental results provide a clear basis for assessing the extent to which the research objectives stated in Chapter One have been achieved.

First, the objective of developing a comprehensive intelligent project planning system was substantially achieved.
The implemented framework successfully transforms raw software requirements into multiple structured planning artifacts, including task plans, dependency graphs, sprint summaries, critic reports, and risk reports. The system also exposes these outputs through an operational API and web interface, confirming that the work progressed beyond theoretical modeling into a functional software prototype. This outcome is aligned with recent directions in LLM-assisted software engineering and requirements processing [3], [4], [7], [8].

Second, the objective of investigating and comparing alternative AI planning configurations was achieved.
The evaluation framework supports multiple execution modes, including `rules`, `rules_kb`, `llm`, `llm_kb`, and `zero_shot` conditions. This allows the project to compare deterministic planning, retrieval-augmented planning, and LLM-based planning under controlled benchmark conditions. The results show that the hybrid architecture improves structural planning quality over the rule-only baseline, even though direct LLM reliability remains limited. This comparison is methodologically relevant because retrieval-augmented generation and multi-agent LLM systems are commonly evaluated by comparing complete and reduced configurations [2], [4], [16], [17].

Third, the objective of performing quantitative and qualitative evaluation was clearly achieved.
The proposed model was evaluated on 20 benchmark samples using a multi-metric framework that included coverage, dependency coherence, critic score, MMRE, PRED(25), F1-FR, F1-NFR, complexity entropy, and the aggregate overall score. In addition, the system supports visual inspection through dashboard, graph, and evaluation interfaces, thereby combining numerical evaluation with practical interpretability. The use of effort estimation metrics such as MMRE and PRED(25) is supported by software effort estimation literature [15], [18], [36], while the use of coverage and requirement-type assessment is aligned with requirements engineering practice [38].

Fourth, the objective of balancing quality with practical deployability was only partially achieved.
On the positive side, the system achieved **100% benchmark pass coverage**, operated

entirely in a local-first environment, and produced highly coherent final outputs. However, the threshold assessment also revealed important limitations: the **fallback rate of 75.0%** exceeded the desired operational threshold, the **direct-generation rate of 25.0%** remained well below the target level, and the **average overall score of 0.770** fell slightly below the internal target of **0.78**. Accordingly, while the system is already deployable as a robust hybrid framework, the current LLM component still requires further refinement if fully autonomous planning performance is to become a primary strength rather than a secondary capability.

Overall, the evidence suggests that the research objectives were **largely achieved at the system and methodology level**, with the main remaining limitation lying in the **stability of direct local LLM planning**. This is a meaningful outcome rather than a weakness alone, because it confirms the value of the proposed hybrid design: even when the LLM layer is imperfect, the broader architecture preserves reliability and maintains usable planning quality. This conclusion supports the architectural rationale of combining LLM-guided reasoning, retrieval-augmented context, deterministic fallback, and structured validation in software planning systems [2], [3], [4], [7], [8], [16], [17].

5.8 Comparison and Analysis of Results

The quantitative comparisons below are derived from evaluation_report.json, evaluation_llm_kb_report.json, ablation_report.json, and the versioned reports in data/evaluation.

This section analyzes the behavior of the proposed framework from four complementary perspectives: internal ablation, developmental evolution, analytical comparison with previous approaches, and a consolidated discussion of strengths, limitations, and improvement opportunities. The purpose is not only to report performance values, but also to explain what these values reveal about the design choices adopted in AI PM. This type of multi-dimensional interpretation is consistent with software project management and effort-estimation literature, where planning quality is usually evaluated through several complementary indicators rather than a single isolated metric [15], [18], [36], [37].

5.9 Comparison Between the Proposed Configurations

5.9.1 Ablation Study: Rules, Rules+KB, Rules+KB+LLM

To understand the contribution of each architectural layer, the system was analyzed under three main configurations: a pure rule-based baseline, a rule-based configuration with knowledge-base calibration, and the full hybrid pipeline combining rules, KB support, and LLM reasoning. This ablation-oriented comparison is methodologically useful because it separates the contribution of deterministic rules, retrieval-augmented knowledge, and LLM-based reasoning [2], [3], [4], [16], [17].

Configuration	Pass Rate	Overall Score	Dependency Coherence	MMRE	PRED(25)	F1-FR	F1-NFR	Direct Generation	Fallback Rate
Rules	100.0%	0.749	0.985	0.271	0.777	0.563	0.452	0.0%	100.0%
Rules + KB	100.0%	0.749	0.985	0.255	0.755	0.563	0.452	0.0%	100.0%
Rules + KB + LLM	100.0%	0.770	0.990	0.301	0.691	0.573	0.482	25.0%	75.0%

The ablation study shows three clear effects. First, adding the **knowledge base alone** did not change the structural metrics, since the current rule-based planner does not use KB retrieval for decomposition; instead, the KB mainly calibrates effort estimation. This explains why the **Overall Score remained unchanged at 0.749**, while **MMRE improved slightly from 0.271 to 0.255**. The use of retrieved historical or semantically similar records for calibration is consistent with retrieval-augmented reasoning and software estimation approaches that rely on historical references or project-specific calibration [2], [6], [15], [18], [36].

Second, adding the **LLM layer** improved the planning-oriented dimensions of the system. The full hybrid model increased the **Overall Score to 0.770**, improved **dependency coherence** from **0.985** to **0.990**, and raised both **F1-FR** and **F1-NFR**. This indicates that the LLM contributed primarily to **semantic planning quality**, especially in task interpretation and requirement-type handling. This observation is aligned with recent work on LLM-assisted software engineering and requirements processing [3], [4], [7], [8].

Third, the hybrid condition also revealed the current limits of the local LLM layer. Although the final accepted outputs were strong, the model still required fallback in **75.0%** of the benchmark cases, and its estimation quality became less stable than the deterministic baseline. Therefore, the ablation results support the following interpretation: **the LLM adds planning intelligence and structural expressiveness, whereas the rule-based and KB layers remain essential for stability and estimation discipline**. This supports the broader rationale for hybrid and multi-agent LLM systems, where generative reasoning is combined with validation, control, and specialized supporting modules [4], [16], [17].

5.9.2 Evolution of System Performance Across Development Versions

V2 to V12

The versioned evaluation reports provide a useful picture of how the system matured during development. Rather than improving monotonically in every metric, the system evolved through a trade-off between **autonomy**, **stability**, and **planning quality**. This kind of iterative evaluation supports reproducible experimental development and transparent reporting of model behavior across versions [37], [38].

Version	Pass Rate	Direct Generation	Fallback Rate	Overall Score	MMRE	F1-FR	F1-NFR
V2	90.0%	35.0%	65.0%	0.764	0.341	0.640	0.508
V6	100.0%	25.0%	75.0%	0.771	0.334	0.593	0.475
V8	100.0%	65.0%	35.0%	0.765	0.346	0.712	0.452
V10	100.0%	70.0%	30.0%	0.766	0.344	0.712	0.420
V11	100.0%	65.0%	35.0%	0.788	0.360	0.705	0.470
V12	100.0%	75.0%	25.0%	0.787	0.359	0.705	0.470

The early versions, `V2–V6`, prioritized structural correctness and benchmark completion, but relied heavily on fallback execution. During the middle stage, `V7–V10`, the system became substantially more autonomous, with direct generation rising from **35.0%** in `V2` to **70.0%** in `V10`. However, this increase in autonomy briefly introduced modest instability, as seen in the **95.0%** pass rate of `V7` and `V9`, together with lower dependency quality in those versions.

The late-stage versions, `V11–V12`, showed the most balanced behavior. `V11` achieved the **highest overall score, 0.788**, while `V12` delivered the **best operational balance**, combining **100.0% pass rate**, **75.0% direct generation**, and a reduced **25.0% fallback rate**. At the same time, classification performance improved meaningfully relative to early versions, with the average classification score rising from **0.613** in earlier releases to **0.685** in the final stage.

These results suggest that the project's development trajectory was successful in one important sense: later versions became more autonomous without collapsing the structural strengths of the framework. However, the results also show that greater LLM autonomy did not automatically improve effort-estimation quality, which remained one of the most challenging dimensions throughout development. This is consistent with the broader software effort estimation literature, where effort prediction remains sensitive to calibration, dataset quality, and model assumptions [6], [14], [15], [18], [36].

5.9.3 Comparison with Previous Approaches

COCOMO II, Desharnais Regression, AutoGen, MetaGPT, CrewAI

This comparison should be interpreted as **analytical rather than strictly experimental**, because the external approaches were not executed on the same 20-sample benchmark under identical conditions. Moreover, **COCOMO II** and **Desharnais regression** are estimation-oriented baselines, whereas **AutoGen**, **MetaGPT**, and **CrewAI** are agent frameworks rather than specialized requirements-to-plan systems. COCOMO II is a classical parametric effort-estimation model [27], [34], while historical-data-based estimation approaches are widely discussed in software effort estimation literature [6], [15], [18], [36]. AutoGen, MetaGPT, and CrewAI represent general multi-agent or agent-orchestration approaches rather than project-planning-specific evaluation frameworks [16], [17], [30].

Approach	Main Focus	Strength Relative to AI PM	Limitation Relative to AI PM
COCOMO II	Parametric software effort estimation	Stronger for coarse-grained cost estimation discipline and formula transparency	Cannot parse raw natural-language requirements, generate task graphs, classify FR/NFR, or produce sprint/risk artifacts
Desharnais Regression	Historical-data-driven effort prediction	Simple and statistically grounded estimation baseline	Requires structured project attributes and offers no decomposition, dependency reasoning, or validation layer
AutoGen	General-purpose multi-agent orchestration	More flexible as a reusable agent-conversation framework	Does not provide domain-specific planning metrics, deterministic task validation, or a benchmarked requirements-to-plan pipeline
MetaGPT	SOP-style multi-agent software production	Stronger role formalization for broader software artifact generation	Targets full software production more than validated planning from raw requirements; lacks AI PM's planning-specific evaluation structure
CrewAI	Agent workflow orchestration for production use	Good operational orchestration and extensibility	Framework-level capability only; it does not natively solve FR/NFR planning, dependency analytics, or planning-quality evaluation
AI PM	Hybrid requirements-to-plan pipeline	Stronger specialization, traceability, graph analytics, critic validation, and benchmark-driven evaluation	Less general than agent frameworks and still dependent on fallback for stable local LLM execution

From this comparison, three points become clear. First, **traditional estimation models** such as COCOMO II and Desharnais regression remain valuable references for numerical effort prediction, but they address only one subproblem of the planning lifecycle. This limitation is consistent with the distinction between effort estimation models and broader project planning processes [15], [18], [34], [36]. Second, **general multi-agent frameworks** such as AutoGen, MetaGPT, and CrewAI provide orchestration capability, but they do not inherently provide the planning-specific validation, dependency logic, and evaluation discipline implemented in AI PM [16], [17], [30]. Third, the distinctive contribution of AI PM lies in its **specialization**: it is narrower than general frameworks, but stronger as a research-oriented system for converting raw software requirements into validated planning artifacts.

5.9.4 Analysis of Strengths, Weaknesses, and Improvement Directions

The experimental evidence indicates that AI PM has several important strengths. The system achieved **100% benchmark pass rate** in its stable configurations, maintained **perfect requirement coverage**, and consistently produced **high critic scores** and **strong dependency coherence**. Its hybrid architecture also proved highly robust, since fallback logic prevented LLM failure from turning into pipeline failure. In addition, the local-first design, vector knowledge support, and explicit evaluation framework make the system unusually suitable for academic and privacy-sensitive settings. These strengths are consistent with the value of combining LLM-assisted reasoning, retrieval-augmented grounding, multi-agent validation, and structured requirements analysis [2], [3], [4], [7], [8], [16], [17], [38].

At the same time, several weaknesses remain visible. The most important is the **continued reliance on fallback execution** in the main hybrid evaluation, which shows that the local LLM is not yet sufficiently reliable as a standalone planner. A second weakness is the **instability of effort-estimation quality** once the LLM layer becomes more active; the best structural versions were not always the best estimation versions. A third limitation lies in **NFR classification variability**, which improved over time but remained less stable than FR handling. These limitations are relevant because software requirements engineering requires stable handling of both functional and non-functional concerns [38], while effort estimation remains sensitive to calibration quality and historical reference data [6], [14], [15], [18], [36].

These findings suggest several concrete improvement directions. The first is to strengthen the local planning model through better prompt control, narrower output schemas, or domain-adapted fine-tuning. The second is to expand the knowledge base with more **task-level estimation records** and richer **non-functional planning examples**, especially for performance, security, and compliance constraints. The third is to adopt a more selective hybrid policy in which the LLM is used mainly for semantic decomposition and classification, while estimation remains more strongly regularized by deterministic and retrieval-based signals. Finally, a larger benchmark suite and direct reimplementation of external baselines would further strengthen the empirical position of the work.

Overall, the comparison and analysis support a clear conclusion: **AI PM is strongest as a trustworthy hybrid planning framework, not as a purely generative planner**. Its main value lies in combining AI-assisted reasoning with explicit validation, graph-based execution logic, and reproducible evaluation, while its main future opportunity lies in reducing the current dependence on fallback without sacrificing those structural strengths. This conclusion is aligned with the broader direction of hybrid AI-based software engineering systems and multi-agent LLM architectures [3], [4], [16], [17].

5.10 Sources for the comparative frameworks

You do not need this bullet list if these sources are already included in your final IEEE reference list. Instead, keep only the numbered IEEE citations inside the text.

- COCOMO II: [27], [34]
- Desharnais / historical effort estimation: [6], [15], [18], [36]
- AutoGen: [16]
- MetaGPT: [17]
- CrewAI: [30]
- Requirements Engineering / traceability / FR-NFR distinction: [38]
- RAG / vector knowledge grounding: [2]

6 Chapter six : Conclusion

6.1 Introduction

This chapter concludes the thesis by synthesizing the main findings of the study, reflecting on the extent to which the proposed research objectives were achieved, and outlining the principal challenges and future directions emerging from the development and evaluation of the proposed AI PM framework. The thesis set out to address a persistent gap in software engineering practice and research: the lack of a trustworthy, integrated, and practically deployable system capable of transforming raw natural-language software requirements into structured project-planning artifacts. In response, the study proposed and implemented a hybrid AI-driven planning framework that combines local LLM reasoning, retrieval-augmented knowledge support, deterministic validation, graph-based dependency modeling, effort estimation, sprint planning, and risk-aware analysis within a single coherent pipeline. This direction is aligned with recent research on LLM-assisted software engineering, requirements engineering, retrieval-augmented generation, and multi-agent LLM systems [2], [3], [4], [7], [8], [16], [17], [38].

The research demonstrated that intelligent project planning should not be treated as a single-output text-generation task. Rather, it requires a multi-stage process in which requirement interpretation, task decomposition, structural verification, execution modeling, and empirical evaluation are all tightly integrated. Through the design and implementation of AI PM, this thesis contributes both a functional software system and a methodological perspective on how hybrid AI can be used in software planning in a manner that is more reliable, auditable, and operationally meaningful than purely manual or purely generative alternatives. This position is consistent with the broader view that software planning and estimation require structured reasoning, traceability, measurable evaluation, and project-management control rather than isolated prediction or generation alone [15], [18], [36], [37], [38].

6.2 Conclusions

The findings of this thesis support several major conclusions.

First, the study confirms that natural-language software requirements can be transformed into coherent planning artifacts through a hybrid intelligent pipeline. The proposed system was able to convert project briefs and requirement templates into structured task plans, dependency graphs, sprint-oriented execution views, critic reports, and risk summaries. This demonstrates that the transition from unstructured requirement text to actionable planning outputs can be meaningfully automated when language reasoning is combined with validation and structural modeling. This conclusion is aligned with recent studies on LLM-supported requirements structuring and requirements specification generation [7], [8].

Second, the results show that hybrid architecture is more suitable for this problem than a purely generative approach. The experimental evidence indicates that the combination of LLM reasoning, retrieval-augmented grounding, and deterministic fallback logic produced

reliable end-to-end behavior. In particular, the framework maintained full benchmark completion and high structural quality even when the local LLM alone was not consistently reliable. This validates the central architectural hypothesis of the thesis: in software project planning, the most effective role of the LLM is not to replace formal planning logic, but to augment it within a constrained and reviewable system. This conclusion is consistent with the literature on multi-agent LLM systems and LLM-assisted software engineering, where role separation, coordination, and validation improve reliability [3], [4], [16], [17].

Third, the proposed system demonstrated strong performance in the structural dimensions of planning quality. Across benchmark evaluation, the framework achieved complete requirement coverage, high dependency coherence, and a very strong critic score in the primary hybrid condition. These results indicate that the system is especially effective at preserving traceability, producing execution-valid dependency structures, and avoiding structurally weak or inconsistent plans. Therefore, one of the clearest contributions of the project lies in its ability to produce planning artifacts that are not only generated, but also verifiable and operationally interpretable. The importance of traceability, structural validity, and analyzable requirements artifacts is supported by requirements engineering standards [38], while dependency and critical-path reasoning are grounded in project scheduling methods [33], [37].

Fourth, the results reveal that planning quality is multidimensional and cannot be adequately represented by a single metric. The use of structural metrics, estimation metrics, classification metrics, and an aggregate overall score proved necessary for capturing the real behavior of the system. This supports the methodological contribution of the thesis, namely that intelligent planning systems should be evaluated through a balanced framework combining semantic completeness, structural integrity, complexity realism, and quantitative estimation behavior. This is consistent with software effort estimation and project management literature, where planning quality and estimation performance are typically evaluated through multiple complementary indicators [15], [18], [36], [37].

Fifth, the study shows that retrieval-augmented support contributes more clearly to calibration and contextual grounding than to raw structural generation alone. In the current implementation, the vector knowledge base was particularly useful for effort calibration and contextual support, while most structural gains emerged from the interaction between reasoning, validation, and graph analysis. This suggests that retrieval is most valuable when used to stabilize and contextualize AI planning rather than as a substitute for planning logic. This conclusion is aligned with retrieval-augmented generation research, where retrieved knowledge supports grounding and improves the reliability of generated outputs [2].

Finally, the thesis concludes that AI PM achieved its research objectives to a substantial degree. The system was successfully implemented, experimentally evaluated, and shown to function as a complete intelligent planning environment. At the same time, the findings remain appropriately critical: the local LLM layer still exhibited a significant dependence on fallback behavior, and some quantitative metrics, particularly those related to effort estimation and non-functional classification stability, remain less mature than the structural planning layer. Thus, the project should be understood as a strong and validated research prototype with clear practical promise, rather than as a finished autonomous planning product.

6.3 Challenges and Future Directions

Despite the success of the proposed framework, the study encountered several important challenges that also define the future research agenda.

One of the most significant challenges was the stability of local LLM planning under structured output constraints. Although the final hybrid system produced high-quality accepted outputs, the model frequently required fallback support when dealing with strict decomposition, cardinality preservation, and JSON-constrained planning. This indicates that local LLMs, while highly promising, still require stronger control mechanisms when applied to formal planning tasks. Future work should therefore investigate improved prompt strategies, schema-aware decoding, constrained generation techniques, and domain-adapted fine-tuning in order to reduce reliance on fallback execution. This direction is supported by the broader literature on LLM reasoning, prompting, and LLM-based software engineering systems [1], [3], [4], [31], [32].

A second challenge concerns the difficulty of achieving uniformly strong estimation performance across diverse software domains. While the project achieved useful effort predictions and meaningful calibration, the estimation layer remained more variable than the structural planning layer. Future work could address this by expanding the task-level estimation corpus, enriching the vector knowledge base with more domain-specific calibration examples, and integrating stronger historical regression or learning-based estimation models alongside the current hybrid estimator. This direction is consistent with software effort estimation research, which emphasizes the importance of calibration, historical data quality, and model selection [6], [14], [15], [18], [34], [36].

A third challenge lies in the classification and treatment of non-functional requirements, especially when such requirements are expressed ambiguously or embedded within broader narrative descriptions. Although the system improved over successive versions, NFR handling remained less stable than FR decomposition in some benchmark conditions. Future research could focus on specialized NFR reasoning modules, better quality-attribute taxonomies, and more explicit conflict handling between functional and non-functional planning constraints. This is aligned with requirements engineering practice, where functional and non-functional requirements require clear identification, classification, and traceability [38].

Another important challenge is the limited scope of comparative benchmarking against external systems. While this thesis provided strong internal evaluation and analytical comparison with classical and modern approaches, future work would benefit from direct execution-based benchmarking against broader planning baselines, additional agentic systems, and alternative local or cloud-based language models. This would strengthen the empirical positioning of the framework within the wider software-engineering AI landscape. Such future benchmarking could include traditional estimation approaches such as COCOMO II and historical-data-driven estimation, as well as multi-agent frameworks such as AutoGen, MetaGPT, and CrewAI [16], [17], [27], [30], [34].

From an applied perspective, future work should also extend the framework toward human-in-the-loop refinement and user-centered validation. Although the system already supports

review, dashboard inspection, and monitoring, further evaluation with students, supervisors, project managers, or industrial practitioners would provide valuable insight into usability, trust, and real-world planning utility. Such studies would help determine not only whether the generated plans are structurally strong, but also whether they improve human decision-making in practice. This direction is consistent with project management principles that emphasize monitoring, stakeholder review, and practical decision support [37].

Additional future directions include:

expanding the benchmark dataset beyond the current 20-sample evaluation set,

incorporating richer project-history signals into monitoring and progress interpretation,

improving sprint-capacity modeling with team-specific resource constraints,

integrating learning-based risk prediction over historical task graphs,

and supporting iterative plan revision through feedback-aware multi-agent interaction.

In a broader sense, the future of this research lies in moving from hybrid AI-assisted planning toward adaptive, continuously improving planning ecosystems in which retrieval, validation, reasoning, monitoring, and human feedback work together over the full project lifecycle. The present thesis establishes a strong foundation for that direction by showing that trustworthy AI-based project planning is both technically feasible and methodologically defensible when designed as a structured, hybrid, and evaluation-driven system. This final direction reflects the intersection of LLM-assisted software engineering, multi-agent collaboration, retrieval-augmented reasoning, and project management practice [2], [3], [4], [16], [17], [37].

7 References

[1] A. Vaswani et al., "Attention Is All You Need," in Advances in Neural Information Processing Systems (NeurIPS), 2017. [Online]. Available: https://arxiv.org/abs/1706.03762

[2] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems (NeurIPS), 2020. [Online]. Available: https://arxiv.org/abs/2005.11401

[3] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, and J. M. Zhang, "Large Language Models for Software Engineering: Survey and Open Problems," in 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE), 2023, pp. 31–53, doi: 10.1109/ICSE-FoSE59343.2023.00008.

[4] J. He, C. Treude, and D. Lo, "LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead," arXiv preprint, 2024 (rev. v4, Jul. 2025). [Online]. Available: https://arxiv.org/abs/2404.04834

[5] S. Basu, M. H. Kim, S. Tatlidil, T. Williams, S. Sloman, and R. I. Bahar, "Augmenting Large Language Models with Psychologically Grounded Models of Causal Reasoning for Planning under Uncertainty," Frontiers in Artificial Intelligence, vol. 8, Jan. 2026, doi: 10.3389/frai.2025.1730614.

[6] O. R. Yürüm, H. Ünlü, and O. Demirörs, "An Alternative Software Benchmarking Dataset: Effort Estimation with Machine Learning," Journal of Systems and Software, vol. 231, 2026, Art. no. 112591, doi: 10.1016/j.jss.2025.112591.

[7] J. J. Norheim and E. Rebentisch, "Structuring Natural Language Requirements with Large Language Models," in 2024 IEEE International Requirements Engineering Workshop (REW), 2024, doi: 10.1109/REW61692.2024.00013.

[8] T. Zhu, L. C. Cordeiro, and Y. Sun, "REQINONE: A Large Language Model-Based Agent for Software Requirements Specification Generation," in 33rd IEEE International Requirements Engineering Conference (RE), 2025, doi: 10.1109/RE63999.2025.00054.

[9] E. Albaroudi, M. Hatamleh, S. M. Hejazi, A. Y. Alshalabi, T. Mansouri, and A. Alameer, "COLLAB-LLM: A Communication-Centric Role-Based Framework for Scalable Multi-

Agent LLM Collaboration," Asian Journal of Research in Computer Science, 2026. [Online]. Available: https://journalajrcos.com/index.php/AJRCOS/article/view/811

[10] M. U. Zeshan, M. Ibiyo, C. Di Sipio, P. T. Nguyen, and D. Di Ruscio, "Many Hands Make Light Work: An LLM-based Multi-Agent System for Detecting Malicious PyPI Packages (LAMPS)," arXiv preprint, Jan. 2026. [Online]. Available: https://arxiv.org/abs/2601.12148

[11] H. Cheng, M. Kim, F. Khomh, T. Racharak, N. Yoshioka, N. Ubayashi, and H. Washizaki, "QUARE: Multi-Agent Negotiation for Balancing Quality Attributes in Requirements Engineering," arXiv preprint, Mar. 2026. [Online]. Available: https://arxiv.org/abs/2603.11890

[12] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A Comprehensive Survey on Graph Neural Networks," IEEE Transactions on Neural Networks and Learning Systems, vol. 32, no. 1, pp. 4–24, 2021, doi: 10.1109/TNNLS.2020.2978386.

[13] Z. Zhang, H. Li, Z. Zhang, Y. Qin, X. Wang, and W. Zhu, "Graph Meets LLMs: Towards Large Graph Models," arXiv preprint, 2023. [Online]. Available: https://arxiv.org/abs/2308.14522

[14] M. Azzeh, D. Nassif, and S. Banitaan, "Predicting Software Effort from Use Case Points: A Systematic Review," Science of Computer Programming, vol. 204, 2021, Art. no. 102596, doi: 10.1016/j.scico.2020.102596.

[15] M. Jørgensen and M. Shepperd, "State of the Practice in Software Effort Estimation: A Survey and Literature Review," arXiv preprint, 2014. [Online]. Available: https://arxiv.org/abs/1401.5878

[16] Q. Wu et al., "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," arXiv preprint, 2023. [Online]. Available: https://arxiv.org/abs/2308.08155

[17] S. Hong et al., "MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework," arXiv preprint, 2023. [Online]. Available: https://arxiv.org/abs/2308.00352

[18] R. Jeffery, M. Ruhe, and I. Wieczorek, "A Comparative Study of Two Software Development Cost Modeling Techniques Using Multi-Organizational and Company-Specific

Data," Information and Software Technology, vol. 42, no. 14, pp. 1009–1016, 2000, doi: 10.1016/S0950-5849(00)00153-1.

[19] Ollama, "Ollama Documentation." [Online]. Available: https://docs.ollama.com/ [Accessed: May 3, 2026].

[20] Chroma, "Chroma Documentation." [Online]. Available: https://docs.trychroma.com/ [Accessed: May 3, 2026].

[21] NetworkX, "NetworkX Documentation." [Online]. Available: https://networkx.org/documentation/stable/ [Accessed: May 3, 2026].

[22] FastAPI, "FastAPI Documentation." [Online]. Available: https://fastapi.tiangolo.com/ [Accessed: May 3, 2026].

[23] React, "React Documentation." [Online]. Available: https://react.dev/ [Accessed: May 3, 2026].

[24] Vite, "Vite Guide." [Online]. Available: https://vite.dev/guide/ [Accessed: May 3, 2026].

[25] Sentence Transformers, "Sentence Transformers Documentation." [Online]. Available: https://sbert.net/ [Accessed: May 3, 2026].

[26] International Function Point Users Group (IFPUG), "Function Point Analysis (FPA)." [Online]. Available: https://ifpug.org/ifpug-standards/fpa [Accessed: May 3, 2026].

[27] Boehm Center for Systems and Software Engineering, "COCOMO II." [Online]. Available: https://boehmcsse.org/tools/cocomo-ii/ [Accessed: May 3, 2026].

[28] Atlassian, "Jira Planning Solutions." [Online]. Available: https://www.atlassian.com/jira/solutions/planning [Accessed: May 3, 2026].

[29] ClickUp, "What is ClickUp Brain?" [Online]. Available: https://help.clickup.com/hc/en-us/articles/12578085238039-What-is-ClickUp-Brain [Accessed: May 3, 2026].

[30] CrewAI, "CrewAI Documentation." [Online]. Available: https://docs.crewai.com/ [Accessed: May 3, 2026].

ثالثاً: المراجع الجديدة — أبحاث، كتب، ومعايير دولية

[31] T. B. Brown et al., "Language Models are Few-Shot Learners," in Advances in Neural Information Processing Systems (NeurIPS), 2020. [Online]. Available: https://arxiv.org/abs/2005.14165

[32] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in Advances in Neural Information Processing Systems (NeurIPS), 2022. [Online]. Available: https://arxiv.org/abs/2201.11903

[33] J. E. Kelley and M. R. Walker, "Critical-Path Planning and Scheduling," in Proc. Eastern Joint IRE-AIEE-ACM Computer Conference, 1959, pp. 160–173, doi: 10.1145/1460299.1460318.

[34] B. W. Boehm, Software Engineering Economics. Englewood Cliffs, NJ, USA: Prentice-Hall, 1981. ISBN: 0-13-822122-7.

[35] K. Schwaber and M. Beedle, Agile Software Development with Scrum. Upper Saddle River, NJ, USA: Prentice Hall, 2002. ISBN: 0-13-067634-9.

[36] S. D. Conte, H. E. Dunsmore, and V. Y. Shen, Software Engineering Metrics and Models. Redwood City, CA, USA: Benjamin/Cummings, 1986. ISBN: 0-8053-2162-4.

[37] Project Management Institute, A Guide to the Project Management Body of Knowledge (PMBOK Guide), 7th ed. Newtown Square, PA, USA: PMI, 2021. ISBN: 978-1-628-25664-2.

[38] Systems and Software Engineering — Life Cycle Processes — Requirements Engineering, ISO/IEC/IEEE Standard 29148:2018, 2018. [Online]. Available: https://www.iso.org/standard/72089.html